

Abstract

Post-quantum cryptography standards have been published, but organisations face a practical question with no empirical answer in the existing literature: what does it actually cost, in latency and storage, to integrate these algorithms into a complete cloud file-signing workflow rather than a standalone benchmark? This study fills this gap by implementing and benchmarking a security-hardened Rust microservice prototype that models a more realistic backend signing pipeline comprising an API gateway, a dedicated hashing service, a manifest builder, a PostgreSQL database, and a MinIO object store.

A formal Phase 1 campaign executes 63 000 measured runs across 630 conditions - seven signature profiles (RSA-PSS, ECDSA, EdDSA, HMAC-SHA256, ML-DSA, SLH-DSA, FN-DSA), three hash algorithms (SHA-256, BLAKE3, Keccak-256), five payload sizes (10 KB-50 MB), and six workflow scenarios - to produce decision-grade performance data at 95% bootstrap confidence [14]. The central finding is that FN-DSA (Falcon) and ML-DSA (Dilithium), both NIST-standardised post-quantum signature schemes impose less end-to-end latency than RSA-PSS across all payload sizes; FN-DSA, in particular, achieves faster signing than ECDSA and the fastest public-key verification in the study at 0.25 ms. SLH-DSA’s 1.6-second signing cost renders it unsuitable for interactive cloud signing paths, though its conservative hash-based security model may suit low-rate archival or root-certificate scenarios. Hash algorithm choice is a secondary concern at small files; BLAKE3 outperforms SHA-256 by 7% at 50 MB, while Keccak-256 trails both by 35% with no compensating advantage.

These results identify the carry-forward configurations for a planned Phase 2 hybrid benchmarking campaign, in which the best-performing classical and post-quantum profiles are paired in dual-signature configurations that model the transitional strategy recommended in the NIST migration guidance document. Crucially, the central finding challenges the prevailing assumption that PQC migration entails a performance-security trade-off: for the two lattice-based NIST standards, the trade-off does not exist at the system level. This dissertation presents an open, fully reproducible benchmarking harness and a dataset of 63,000 system-level measurements to support evidence-based PQC migration decisions in cloud file-handling infrastructure.

Acknowledgements

I would like to thank my supervisor, peers, friends, and family for their support throughout this project.

Their feedback, patience, and encouragement helped shape both the implementation and the dissertation.

Contents

Acknowledgements	ii
1 Introduction	1
1.1 Problem Statement	1
1.2 Research Questions	2
1.3 Aims and Objectives	2
1.4 Scope	3
1.5 Limitations	3
1.6 Report Structure	4
2 Literature Review	5
2.1 The Quantum Threat and Threat Models	5
2.1.1 Harvest Now, Decrypt Later Risk	6
2.1.2 Current State of the Cryptographic Standards	6
2.2 NIST PQC Standardisation	7
2.3 Post-Quantum Algorithm Families	7
2.3.1 ML-KEM and ML-DSA - Lattice-Based Signatures	8
2.3.2 SLH-DSA - Hash-Based Signature	8
2.3.3 FN-DSA - NTRU-Based Signature	9
2.4 File Handling in Cloud	9
2.5 Related Work in System-Level Performance Verification	11
2.6 Summary and Research Gaps	12
3 Methodology	13
3.1 Research Design and Approach	13
3.2 Workflow Design	14
3.3 Storage Backend	15
3.4 Manifest Design	16
3.5 Microservice Architecture Decision	16
3.6 User Interface	17
3.7 Verification Service Design	18
3.8 Security Hardening and Threat Modelling	19
3.9 Algorithm Selection	20
3.9.1 Phase 1: Baseline Establishment	21
3.9.2 Hash Algorithms	21
3.9.3 Phase 2: Hybrid Benchmarking	21
3.10 Benchmarking Framework Design	22
3.10.1 Experimental design	22
3.10.2 Warm-up runs	22

3.10.3	Measured runs	23
3.10.4	Seeded randomisation	23
3.10.5	Inter-run delay	23
3.10.6	Storage state modes	23
3.10.7	Campaign orchestration	23
3.11	Dataset Design	24
3.12	Telemetry and Metrics Collection	24
3.12.1	Client-side timings	25
3.12.2	Server-side timings	25
3.12.3	Artefact metrics	25
3.13	Statistical Analysis Methodology	26
3.13.1	Central tendency	26
3.13.2	Dispersion	26
3.13.3	Confidence intervals	26
3.13.4	Ratio analysis	26
3.13.5	Quality gates	27
4	Experimental Results and Analysis	28
4.1	Campaign Provenance and Quality	28
4.2	Hash Algorithm Performance	29
4.3	Signing Performance	30
4.3.1	Falcon (FN-DSA) outperforms ECDSA	31
4.3.2	ML-DSA is in the same order of magnitude as ECDSA	31
4.3.3	RSA-PSS is by far the slowest classical signing algorithm	32
4.3.4	SLH-DSA is unusable for online workloads	32
4.4	Verification Performance	32
4.5	Artifact Overhead	33
4.6	End-to-End Latency	35
4.6.1	Finding 1: FN-DSA and ML-DSA are faster than RSA-PSS end-to-end	36
4.6.2	Finding 2: The network and I/O floor dominate at all but the smallest files	36
4.6.3	Finding 3: SLH-DSA imposes prohibitive overhead for interactive use	37
4.7	Cost of Hybrid vs Classical	37
4.8	Bottleneck Analysis	38
4.9	Algorithm Tradeoffs	39
4.10	Primary Performance Analysis	40
4.11	Infrastructure and Scalability	41
4.12	Practical Scenarios	43

4.13	Phase 2 Carry-Forward Selection	43
4.13.1	Hash algorithms	44
4.13.2	Classical signature schemes	44
4.13.3	PQC schemes	45
4.14	Threats to Validity	46
4.14.1	Localhost environment	46
4.14.2	Generalisation to cloud hardware	46
4.14.3	Warm storage state only	47
4.15	Summary	47
5	Discussion	49
5.1	Answering the Research Questions	49
5.1.1	RQ1: What is the performance cost of post-quantum and hybrid signature schemes compared to classical signatures in a complete end-to-end file-processing lifecycle?	49
5.1.2	RQ2: What is the proven end-to-end performance footprint of deploying hybrid signature architectures, and what do these findings show for the viability of hybrid transition strategies?	50
5.1.3	RQ3: What reproducibility criteria must a system-level cryptographic benchmark satisfy, and does the harness developed in this study meet them?	50
5.1.4	RQ4: What are the security implications and trade-offs of integrating PQC and hybrid signatures into an existing backend microservice architecture?	50
5.2	What This Means for Practitioners	51
5.3	Balancing Algorithm Trade-offs	52
5.4	Contributions	53
5.5	Limitations	53
5.6	Future Work	54
5.6.1	Cloud-native deployment	54
5.6.2	Cold-mode benchmarking	54
5.6.3	Higher security parameter sets	54
5.6.4	Key Management Service integration	55
5.6.5	SLH-DSA for archival signing	55
6	Conclusion	56
	Appendices	64
A	Supplementary Submission Material	64

List of Tables

1	Security requirements for quantum-safe cloud-based file workflows	11
2	Formal Phase 1 campaign summary.	29
3	Hash stage latency and throughput	30
4	Server-side signing latency by profile	31
5	Server-side verification latency by profile	33
6	Signature sizes and overhead (SHA-256, sign_only scenario, warm, median).	34
7	Client-observed end-to-end workflow latency	35
8	Ratio of each profile’s end-to-end median to RSA-PSS median (SHA-256, workflow scenario, warm).	36
9	Phase 2 carry-forward selection summary.	46
10	Summary of key findings mapped to research questions.	48

List of Figures

1	Hybrid overhead relative to the fastest configuration at each payload size.	37
2	Server-side time decomposition for hybrid configurations . . .	38
3	Hybrid signature size as a percentage of the protected 10 KB file.	38
4	Signing vs. verification latency per algorithm (10 KB, SHA-256, IQR/2 error bars).	39
5	End-to-end throughput (MiB/s) by profile across payload sizes.	40
6	Quantum Signature contribution mapping out the true payload percentages.	40
7	The cost of hybrid upgrade: classical baseline (muted) plus the added PQC latency (bright cap).	41
8	Theoretical peak signing throughput (signatures per second, single core) extrapolated from measured cryptographic latency.	42
9	Bandwidth tax: total data transferred over 1 million 10 KB requests.	42
10	Scenario $\times$ configuration latency heatmap (10 KB, SHA-256).	43

1 Introduction

Post-quantum cryptographic standards have now been released, but a practical question remains: what does it actually cost, in terms of latency and storage, to integrate these algorithms into a complete cloud file-signing workflow? How do the results differ end-to-end versus a standalone benchmark? Cloud file-handling pipelines are especially vulnerable to quantum threats - signing keys are long-lived, manifests carry legal and compliance weight over multi-year retention periods, which attracts attackers. The “Forge Later” attack model [25, 69] means that an adversary who harvests classically signed records today could retroactively forge provenance once a cryptographically relevant quantum computer becomes available. The fundamental vulnerabilities of this security system are threatened by Shor’s algorithm [71], which solves the integer factorisation and discrete logarithm problems in polynomial time, leaving current public-key cryptography susceptible to a sufficiently capable quantum adversary. Among such widely deployed signature schemes are Rivest-Shamir-Adleman (RSA) [67] and the Elliptic Curve Digital Signature Algorithm (ECDSA) [29].

In response, the National Institute of Standards and Technology (NIST) initiated the Post-Quantum Cryptography (PQC) standardisation programme and, in August 2024, published the first four quantum-resistant standards: FIPS 203 (ML-KEM), FIPS 204 (ML-DSA), FIPS 205 (SLH-DSA), and FIPS 206 (FN-DSA) [54–57]. Yet existing evaluations of these schemes predominantly benchmark primitives in isolation. The focus is often on raw cryptographic performance rather than system-level costs. While such measurements are useful in general cryptography, the fact is that I/O, serialisation and network dominate end-to-end latency in production pipelines. This dissertation addresses that gap by implementing a prototype system and benchmarking it against a matrix of signature and hash algorithm combinations within a complete file-signing workflow. While it is acknowledged that locally run systems cannot reliably replicate the results of the actual cloud infrastructure, the system attempts to simulate the behaviour of one in the “best case” scenario.

1.1 Problem Statement

The few system-level studies that do exist have focused on constrained IoT hardware or TLS handshake overhead. File-handling workflows, on the other hand, have distinct characteristics, including large-scale object persistence, manifest generation, and integrity verification chains.

No controlled comparison of classical-only, PQC-only, or hybrid signing

strategies has been conducted within a single architecture with uniform experimental conditions, a gap that organisations currently need to fill. Quantitative evidence is needed to select a valid, cost-effective migration path. As system pipelines have limited bandwidth and associated costs, the optimal solution would not benefit from assumptions - these must be identified based on evidence.

Furthermore, the hybrid method (combining classical and post-quantum schemes as a transitional measure) remains insufficiently explored to support recommendations for cloud-based signing infrastructure.

1.2 Research Questions

The following research questions guide this dissertation:

- RQ1. What is the performance cost of post-quantum and hybrid signature schemes compared to classical signatures in a complete end-to-end file-processing lifecycle?
- RQ2. What is the proven end-to-end performance footprint of deploying hybrid signature architectures, and what do these findings show for the viability of hybrid transition strategies?
- RQ3. What reproducibility criteria must a system-level cryptographic benchmark satisfy, and does the harness developed in this study meet them?
- RQ4. What are the security implications and trade-offs of integrating PQC and hybrid signatures into an existing backend microservice architecture?

1.3 Aims and Objectives

This project aims to implement a prototype of a security-hardened backend system that can accommodate file signing and verification workflows. This system must support a variety of hashing and signature techniques and use this prototype to assess the efficiency and feasibility of PQC-only and hybrid approaches relative to classical baselines. While a web user interface will be developed and included for demonstration purposes, all benchmarking will be conducted via a Command-Line Interface (CLI) to avoid browser-related performance penalties. Tests inside the benchmarking campaign are conducted in a controlled, reproducible sequence of workflow runs.

To achieve this, the project sets the following objectives:

1. Design an architecture that minimises client-side variability and enforces consistent cryptographic performance.

2. Incorporate support for a variety of signature and hash profiles. Algorithms must include NIST-approved PQC families.
3. Build a structure for canonical manifest generation - they must guarantee deterministic outputs.
4. Integrate mechanisms for file handling on each stage of the workflow.
5. Build and assess a benchmarking harness that runs tests, collecting performance measurements for multiple configurations.

1.4 Scope

This dissertation is scoped to the signing and verification phases of the workflow. The prototype implements a microservice architecture comprising an API gateway that routes user requests, a hashing service, and a manifest build-and-sign service, supported by a PostgreSQL database and a MinIO object store. The deployment is handled by a containerised local environment. Benchmarking is conducted in two phases.

Phase 1 covers classical (RSA-PSS, ECDSA-P256, EdDSA, HMAC-SHA256) and post-quantum (ML-DSA/FIPS 204, SLH-DSA/FIPS 205, FN-DSA/FIPS 206) signature profile categories. These are evaluated with hash algorithms (SHA-256, BLAKE3, Keccak-256). This setup enables the identification of the two best-performing representatives in each category.

Phase 2 uses those selected profiles in hybrid classical-plus-PQC dual-signing configurations. Mechanisms like key encapsulation and transport security are explicitly out of scope for maximised efficiency and a limited project timeline.

1.5 Limitations

Several limitations apply. The benchmarking setup uses Docker Compose, which does not mimic the hardware specifications of a live cloud environment. Because the conditions share the same infrastructure, it is acceptable to maintain comparisons of signature profiles even though absolute latency values are not directly transferable. Hybrid assessment is limited to the classical-PQC pairings chosen in the first-phase performance analysis; full coverage of all possible pairings would expand the campaign to an unmanageable size and length. Based on this constraint, the two-phase design focuses on configurations with the highest statistical viability. The parameter sets are also limited to one representative for each algorithm (ML-DSA-44, SLH-DSA-128s, Falcon-512), corresponding to NIST Security Level 1-2. Higher levels

would increase signature sizes, but the rank ordering is unlikely to change. Formal security analysis is not addressed, as the dissertation’s contribution is empirical performance measurement rather than cryptanalysis. Execution timings are subject to OS scheduler volatility, which is mitigated by 350 ms inter-run delays and randomised run ordering. A sequence of warm-up runs is also presented as an option, but the variability still cannot be eliminated.

1.6 Report Structure

The remainder of the dissertation is organised as follows: Section 2 reviews the relevant literature, spanning from the quantum threat to classical asymmetric cryptography and its specific implications for cloud storage. The literature review also discusses the current limitations of crypto standards, the NIST PQC standardisation process, and its candidates. Finally, it addresses the security requirements of cloud file workflows and the research gap this dissertation aims to fill. Section 3 describes the design, the rationale behind each major architectural decision, the security hardening measures applied before benchmarking, and the complete analysis framework. Section 4 presents the results of the experiments and analysis of the Phase 1 benchmarking campaign along with Phase 2 carry-forward selection derivations. Section 5 discusses the findings and their implications for organisations considering a PQC migration, and Section 6 concludes the dissertation.

2 Literature Review

This section reviews the literature that supports consideration of post-quantum cryptography in cloud file-handling workflows. It first sets the stage by introducing and explaining the quantum threat to classical asymmetric cryptography and the specific issues it poses for cloud storage. It then presents the advancements in tackling this risk, the NIST standardisation process, and the major post-quantum signature algorithms selected for standardisation. Finally, it discusses the security requirements of cloud file workflows and the literature gaps that motivate this dissertation.

2.1 The Quantum Threat and Threat Models

Classical public-key cryptography is a bedrock of many vital functions in cloud workflows for file handling, spanning from secure key establishment to authentication and non-repudiation. Contemporary industry standard schemes are strong enough to withstand brute-force attacks from traditional computers [53]. However, this is not the case for progressing quantum computers. Shor’s algorithm [71], if implemented on a large enough quantum computer - estimated in 2021 to require around 20 million noisy physical qubits [19] - would render such schemes insecure. The timescale for potential factorisation of RSA-2048 was estimated to be approximately 8 hours under those assumptions [19]. Grover’s algorithm [23], on the other hand, reduces the efficiency of symmetric-key encryption primitives such as AES and SHA-2 by providing quadratic acceleration for key search, essentially halving their safety. Nonetheless, it does not eliminate their protection [8, 23]. The reinforcement of the workflows to become quantum-safe must therefore prioritise substituting current asymmetric cryptography. In the meantime, symmetric and hash-based algorithms can be kept with increased security parameters. This motivates the present study’s scope on digital signature schemes.

The urgency of migration depends on factors beyond the capabilities of quantum computing hardware itself. Mosca [44] defined this risk model: an organisation faces a threat today if the sum of the time required for infrastructure migration and the time until a quantum computer arrives exceeds the required shelf life of its protected data. There are still major engineering challenges before today’s noisy intermediate-scale quantum (NISQ) [64] devices can become systems capable of running Shor’s algorithm at a scale that breaks modern cryptographic security. However, the pace of hardware development is rapid enough that standards bodies treat this as a real concern that requires planning.

2.1.1 Harvest Now, Decrypt Later Risk

The "Harvest Now, Decrypt Later" (HNDL) threat model is the key weakness for cloud storage. The data may be intercepted and hijacked mid-transport via a compromised TLS connection, or exfiltrated from breached data warehouses. Malicious parties can store both the ciphertext and its public key today, and once a relevant quantum computer becomes available, derive private keys to retroactively decrypt the stored data [25, 69]. Strictly speaking, HNDL applies to confidentiality guarantees provided by key encapsulation mechanisms (KEMs) and public-key encryption.

Yet, the "Forge Later" or "Retroactive Repudiation" attacks are an equivalent threat to digital signatures. Technically, an adversary cannot "decrypt" a signature; they can forge fresh, valid-looking manifests for falsified data if a signing key is compromised. The repercussions could be disastrous if a long-lived classical-signing key is recovered by a future attacker using Shor's algorithm. They could retroactively generate signatures on historical manifests, which fundamentally erodes the non-repudiation guarantees. Organisations retaining file manifests with digital signatures beyond the anticipated quantum era must treat this as a calculated risk, as the origin of the stored data will be impossible to verify without quantum-resistant schemes.

2.1.2 Current State of the Cryptographic Standards

Standards specify which algorithms may be used for digital signature generation. Standards such as FIPS 186-5 [53] still rely on RSA and ECDSA despite quantum concerns. They offer little to no resistance against quantum threats - Shor's algorithm can trivially break both. This limitation extends to most existing FIPS 140-2 and FIPS 140-3 certified hardware security modules (HSMs) and key management systems (KMS). Regardless, PQC support has become increasingly more available in select vendor firmware [22]. Major cloud providers have also begun deploying PQC features in their services. In April 2025, AWS implemented ML-KEM hybrid post-quantum TLS in AWS Key Management Service (KMS) and AWS Certificate Manager (ACM) [70]. Moreover, Google Cloud added ML-DSA and SLH-DSA signature support to Cloud KMS in February 2025 [12]. Given that the leading providers have made such contributions to their core infrastructure services, PQC is likely to see general adoption in the near future.

The X.509 certificate infrastructure also remains in transition. The IETF LAMPS working group is actively developing PQC certificate specifications, including algorithm identifier formats and certificate profile definitions [35, 36]. Despite major advancements, the large-scale deployment of PQC-signed

certificates by commercial CAs remains limited as of Q2 2026. The industry is still dominated by early commercial product support rather than a broad WebPKI rollout [65].

2.2 NIST PQC Standardisation

In December 2016, NIST announced a public competition to standardise PQC algorithms and invited submissions [49]. There were 82 candidate algorithms proposed across lattice-based, code-based, multivariate and hash-based families [50, 59]. After a year-long evaluation, only 26 algorithms advanced to the second round in January 2019 [50], with further analysis resulting in 7 finalists and 8 alternative candidates announced in July 2020 [51]. The next two years of theoretical and empirical analysis allowed NIST to publish the first batch of post-quantum standards in August 2024 [54–57]:

- **CRYSTALS-Kyber (ML-KEM)** - standardised as FIPS 203 [54]
- **CRYSTALS-Dilithium (ML-DSA)** - standardised as FIPS 204 [55]
- **SPHINCS+ (SLH-DSA)** - standardised as FIPS 205 [56]
- **Falcon (FN-DSA)** - in development as FIPS 206 [57]

CRYSTALS-Kyber is a Key Encapsulation Mechanism (KEM) used to establish a shared secret over an insecure network. Despite being a NIST-recommended algorithm, it is not a digital signature scheme, thus out of scope for the project timeline.

ML-DSA, SLH-DSA, and FN-DSA are digital signature schemes. The transition guidance was published along with the standards [48, 58], and the U.S. National Security Memorandum NSM-10 [27] has set deadlines for federal systems and high-risk sectors to transition to quantum-safe infrastructure.

After the 4th appraisal round, NIST also added HQC to the standards. It is a code-based KEM, a fifth algorithm for PQC encryption, selected in March 2025 [60, 61], while SIKE, which had been a candidate, was broken and withdrawn [11].

2.3 Post-Quantum Algorithm Families

The three standardised signature algorithms span two distinct design families: lattice-based and hash-based. Since they have different security foundations and performance characteristics, their suitability for deployment in bandwidth-sensitive or constrained environments varies. The following subsections survey each family.

2.3.1 ML-KEM and ML-DSA - Lattice-Based Signatures

Lattice-based cryptography underpins ML-KEM and ML-DSA, the two lattice-based NIST standards. Their security relies on the computational hardness of the Module Learning with Errors (MLWE) problem and related challenges, including the Shortest Vector Problem (SVP) and Closest Vector Problem (CVP) [54, 55, 66]. These problems are connected through worst-case-to-average-case reductions, which means that in the case of a randomly chosen average-case MLWE instance being solved, it would mean the ability to solve the hardest worst-case lattice problems [66]. The ML-DSA algorithm builds on the Fiat-Shamir with Aborts framework introduced by Lyubashevsky [34], yielding an efficient signature scheme that does not require computationally intensive tools to generate secret values.

Lattice-based schemes offer three main advantages for deployment:

Strong Security Reductions. A key distinguishing feature of lattice-based PQC schemes over classical alternatives is the nature of their security reductions. ML-KEM and ML-DSA possess strong worst-case-to-average-case reductions [54]: an adversary capable of breaking these schemes on randomly chosen average-case instances would necessarily be capable of solving the hardest known worst-case instances of SVP and CVP. Conversely, classical schemes like RSA and ECDH rely on unproven average-case hardness assumptions without equivalent worst-case reductions, possibly making them more vulnerable to unpredicted algorithmic breakthroughs [54, 55]. For long-term security, it is a much more robust basis for confidence.

Flexibility. ML-KEM and ML-DSA run efficiently across different architectures - from CPUs to cryptographic accelerators.

Quantum Resistance. Reviews by NIST and the research community state that the assumptions behind ML-KEM and ML-DSA remain sound [54, 55]. No known quantum technique poses a threat to the security of these schemes.

Efficient NTT-based arithmetic, moderate signature sizes, and a deterministic signing process - suggest that ML-DSA should deliver competitive signing and verification latency relative to classical elliptic-curve schemes, a prediction that Section 4 evaluates empirically.

2.3.2 SLH-DSA - Hash-Based Signature

SLH-DSA (FIPS 205 [56]) relies on the collision and pre-image resistances of the underlying hash functions (SHA-256 or SHAKE-256, depending on the variant), with no dependence on algebraic hardness assumptions. This builds on earlier ideas and traces to the one-time signatures of Lamport [33] and the tree-based multi-signature construction of Merkle [40]. A key feature that

distinguishes SLH-DSA is its stateless design, unlike earlier stateful hash-based schemes such as XMSS [28] or LMS [38]. Stateful schemes require secure management and synchronisation of signing state to prevent key reuse [52] - a significant operational burden in distributed cloud environments, which SLH-DSA eliminates.

The trade-off for this conservative security model is signature size. SLH-DSA-SHA2-128s (smallest variant) produces signatures of 7856 bytes, compared to ML-DSA-44, which has a higher security category and produces only 2420 bytes [55, 56]. This may be a non-trivial overhead in manifest pipelines and bandwidth-sensitive interfaces. Signing throughput is also worse than for lattice-based algorithms; this combination of large signatures and high signing cost implies that SLH-DSA will impose the heaviest overhead of the three PQC schemes in a throughput-sensitive cloud pipeline - a hypothesis quantified in Section 4.

2.3.3 FN-DSA - NTRU-Based Signature

Falcon [17] is a lattice-based signature scheme constructed over NTRU lattices - a family introduced by Hoffstein, Pipher, and Silverman [26]. Unlike ML-DSA with its rejection sampling, Falcon employs a Gaussian lattice trapdoor sampler over NTRU lattices, achieving remarkably light output: Falcon-512 produces signatures of 666 bytes compared to 2420 bytes for ML-DSA-44 [55]. Being in late-stage development and awaiting final standardisation as FN-DSA in FIPS 206 [57], Falcon completes the initial NIST post-quantum signature portfolio.

However, the compactness comes with implementation difficulty: the Fast Fourier Transform-based Gaussian sampler is numerically sensitive and therefore entails precise floating-point handling to achieve constant-time operation. Hence, from a security perspective, Falcon is significantly more demanding than ML-DSA. However, if one works in a high-throughput pipeline or an API gateway in which bandwidth is typically the bottleneck, then Falcon’s smaller signatures can save considerably more bytes than in isolated benchmarks. Section 4 quantifies this bandwidth advantage within the prototype’s signing workflow.

2.4 File Handling in Cloud

A typical cloud file workflow is built around multiple phases that require integrity verification. The pattern is usually a requirement for an envelope encryption architecture - files are first encrypted locally using a data encryption key (DEK). The DEK itself is then protected by a key encryption key

(KEK), and a cloud key management service (KMS) stores and manages it. Thus, the file is protected by the DEK, and the DEK is protected by the KEK [5, 20].

The workflow begins with the upload phase. The client generates or fetches the DEK and encrypts the file locally; the resulting ciphertext and the DEK encrypted by the KEK are uploaded to object storage (e.g., AWS S3). In this case, DEK is typically embedded in metadata or cached in a separate key store [2, 79].

Then comes the storage phase. The service computes and stores checksums to guarantee data remains unchanged during persistent storage. Modern cloud platforms support a long list of checksum algorithms, which can be used to verify files against silent data corruption and transmission errors [3, 4].

To trace activity, all object access must be recorded in logs. Every interaction must be logged in a structured record that includes relevant timestamps, identities, actions, and affected resources, and that maintains a chain of custody (CoC). In some high-risk environments, the logs themselves are signed to prevent manipulation [30]. Such a mechanism enables organisations to comply with data storage regulations and detect unauthorised data access [1].

In the verification phase, the client retrieves the encrypted object and its wrapped DEK. After providing credentials to the KMS to unwrap the DEK, the files are locally decrypted to recompute their checksums and verify their integrity by comparing them to the stored values. Mismatches must raise an integrity violation alert [18].

Table 1 summarises the functional security baseline, clearly defining how every identified operational requirement translates sequentially into distinct cryptographic implications during PQC migrations.

Table 1: Security requirements for quantum-safe cloud-based file workflows

Requirement	Mechanism	PQC Implications
Confidentiality	DEK wrapped by KEK	KEK derivation using ML-KEM
Integrity	Checksums, manifest signatures	ML-DSA/SLH-DSA signatures
Authenticity	Signatures on metadata and logs	Mandatory PQC signatures
Non-repudiation	Digital signatures	ML-DSA/SLH-DSA signatures
Key Management	HSM/KMS stored keys	KMS with PQC support
Auditability	Immutable signed logs	Logs signed with PQC
Chain of Custody	Custody record signatures	Timestamps, PQC signatures

2.5 Related Work in System-Level Performance Verification

The majority of previous work focuses on raw speed measurements of post-quantum cryptography, generally ranging from primitive signature operations measured in CPU cycles to performance on isolated CPUs. The comprehensive benchmarking by the Open Quantum Safe (OQS) project [74] provides excellent timing data, but intentionally limits its focus to internal algorithm speed. It does not consider the impact of microservice operations such as database writes, network delays, and object store interactions on performance, and therefore does not address many of the questions relevant to real-world adoption. Furthermore, when evaluating the network, studies tend to focus on KEMs within TLS handshakes [62, 73] because the HNLD risk model prioritises protecting confidentiality.

There is thus a significant gap in the research literature regarding the performance of PQC signatures in a distributed cloud environment. Most workflow studies focus on constrained IoT hardware or blockchain networks. When they do study cloud file signing, they tend to ignore the storage amplification effects of large PQC signatures compared to their classic counterparts. This leaves the very relevant questions of bandwidth, database scaling costs, and the economics of storing large PQC manifests unaddressed.

2.6 Summary and Research Gaps

As the literature review shows, the threat posed by quantum computers to classical public-key cryptography under current standards is both real and urgent. The HNDL and “Forge Later” threat models mean that an eventual quantum capability compromises the provenance of data signed today. NIST has standardised a post-quantum signature portfolio across two design families, with a third in the final stages of standardisation. This review highlights four research gaps, each of which motivates a specific research question posed in Section 1.2.

First, no system-level study has measured PQC signature performance across the full workflow. Most published evaluations focus on standalone microbenchmarks or TLS handshakes and do not capture the infrastructure effect that dominates real workflows. This gap, together with the absence of a system-level comparison across the three NIST-standardised PQC signature families, motivates **RQ1**.

Second, no prior work has undertaken a controlled empirical comparison of classical-only, PQC-only, and hybrid signing strategies within a single microservice architecture under uniform experimental conditions. The lack of hybrid performance data in particular leaves organisations without evidence-based guidance for transition planning. This motivates **RQ2**.

Third, the reproducibility apparatus required to generate statistically valid system-level benchmarks of cryptographic schemes in such pipelines remains undocumented. This motivates **RQ3**.

Fourth, the security implications and operational trade-offs of integrating PQC primitives into hardened backend microservice architectures have not been systematically examined. This motivates **RQ4**.

These gaps serve as the basis for the prototype design and benchmarking methodology described in the subsequent sections.

3 Methodology

This section describes the research methodology used for the design, implementation and evaluation of the prototype system. The project followed a continuous, iterative approach to building a prototype, where decisions were refined incrementally at each stage in which components were built, and the architectural suitability was assessed. This section presents the rationale behind each major architectural and experimental decision, along with the trade-offs considered. This answers RQ3 while providing the context for interpreting the results reported in Section 4.

3.1 Research Design and Approach

The research prioritises the construction of a representative system and the generation of empirical results over measuring isolated primitives. The aim was to measure the performance of complete signing and verification workflows. Such a method was chosen because of the research attention on practical results from system-level latency testing. Reliable transferable results cannot be achieved with standalone microbenchmarks.

Development followed an iterative agile method. Typical implementation sequence started with exploring an area, planning an implementation strategy, performing trade-off analysis, selecting an approach, developing a component, conducting tests, identifying limitations, and refining. This loop was repeated across each layer. A containerised deployment (Docker [39]) was used throughout in preference to a live cloud deployment. There are three main reasons why this is viable.

Local setup eliminates variability. A local environment reduces the variability introduced by network topology, isolating measurements from algorithmic and architectural differences and from infrastructure noise.

Eliminates cloud costs at scale. It removes recurring charges for cloud APIs and object storage. A single campaign costs approximately £6, but the full testing campaign requires many independent runs, making an environment at this scale economically essential.

Reproducible containerised deployment. Docker-based containerisation ensures that any user can reproduce the system configuration from the repository without manually installing external dependencies, supporting the reproducibility criterion required by RQ3.

Several alternatives were evaluated before selecting Rust as the implementation language for all backend services. It had to be a language currently used in the development of cloud services and be the trend in contemporary software development. The trade-off analysis was therefore conducted between

Rust, Python, Java, and Go.

Python excels at rapid prototyping, but it inherently lags in execution speed and fine-grained concurrency control, which are key characteristics of a high-throughput microservice pipeline. Java offers better performance and is widely used in distributed enterprise systems. But the Java Virtual Machine introduces volatile garbage-collection spikes that contaminate micro-level timing data. GoLang was developed by Google and is actively explored for refactoring the Google Cloud Platform legacy architecture. It provides excellent backend capabilities, yet its garbage collector still risks jitter. Additionally, its post-quantum cryptographic ecosystem is less mature than Rust’s for the required algorithms.

Therefore, Rust [37] was chosen. Its emerging wider adoption in secure systems programming makes it a highly appropriate fit for building a future-facing prototype. Its zero-cost abstractions make it well-suited to latency-sensitive work. Also, the absence of runtime garbage collection and the associated pauses prevents timing skew. Rust is a type system that also provides built-in compile-time memory safety, reducing the risk of undefined behaviour in systems with multiple services. Finally and crucially, it possesses a mature PQC crate ecosystem, including all the signature schemes required for this project.

3.2 Workflow Design

The prototype’s design is based on literature about how similar real-world systems operate. Major cloud object storage providers’ documentation from the was reviewed - most impactfully AWS S3 object integrity checking [3, 4], AWS KMS server-side and client-side encryption guidance [2, 5], Google Cloud KMS and Yandex Cloud KMS envelope encryption references [20, 79], and the AWS CloudTrail log-file integrity validation specification [1]. The source code of the `aws-s3-integrity-check` tool [18] was examined for insight into existing integrity checkers. MinIO’s documentation [42] confirmed that a self-hosted S3-compatible backend could faithfully model the production API surface.

The reference architecture is consistent across platforms. Files are encrypted locally under a per-object data encryption key (DEK), which is wrapped by a key encryption key (KEK) stored in a cloud KMS - the envelope encryption pattern [5, 20]. The encrypted object and its wrapped DEK are uploaded to object storage. The platform computes and stores a checksum at ingestion [2, 3]. On retrieval, the checksum is recomputed and compared; any mismatch indicates silent data corruption or unauthorised modification [4, 18].

Hash-then-sign design decision. A decision was made to sign the file’s

hash digest instead of the raw file content. Two reasons why: the digest is a fixed-length input regardless of file size. This decouples the signing latency from payload size and eliminates the need to re-read the entire file during verification. Second, signing a cryptographic digest precludes length-extension and partial-file presentation attacks: any modification to the file’s bytes invalidates the signature. This hash-then-sign pattern was adopted as the basis for all subsequent workflow design.

3.3 Storage Backend

Once hash-then-sign was established, the prototype required persistent storage: signed manifests must survive across request boundaries so that a later verification call can retrieve and re-evaluate them. Persistence is required for the end-to-end testing; the sign and verify operations would be independent, and therefore could not guarantee integrity. The objects, therefore, require quick, on-demand retrieval. The target was an S3-compatible object store that emulated the API surface used by production cloud deployments, such as AWS S3 or Google Cloud Storage. This was key to ensuring the performance findings would be transferable to real cloud environments.

MinIO [42] was selected because it supports a fully S3-compatible API - exactly the type of storage used in cloud services. Such a decision ensures that the measured metrics reflect the same shape as would be observed when executed against a production endpoint. The only differences are network latency (eliminated by localhost deployment, as intended) and backend storage media. Deploying MinIO as a Docker container alongside the other services enables the 63,000-run benchmarking scale required for statistical validity without incurring the per-request and egress-bandwidth charges that a metered commercial service would impose at this experimental scale. MinIO’s public repository was maintained as open-source until February 2026 [43].

There was also a need for a database to store manifest metadata. The two most prominent candidates were PostgreSQL and MySQL. While both support standard SQL, MySQL lacks native support for several data types used in the manifest schema, requiring workarounds that PostgreSQL supports natively. Notably, this includes JSON operators [63]. Therefore, PostgreSQL was chosen as the database management system for manifest metadata and audit entries. ACID guarantees ensure that a manifest is never partially written, plus its query interface supports the manifest-lookup operations required for the verification service.

3.4 Manifest Design

To cryptographically guarantee a file’s integrity, a hash and a signature alone are insufficient. The full set of integrity and audit requirements identified in the literature review must be implemented. Moreover, there is no way to determine the hash and signature algorithms used, the file size, the date, or the object’s destination without additional metadata. Therefore, there was a demand for a structured manifest record, which was introduced to carry all this along with the cryptographic material.

The core requirement for the manifest is that it be deterministic: given the matching file and the same algorithm identifiers, the manifest serialisation must produce an identical byte sequence in each independent run. This is necessary for verification flows, as the service reconstructs the manifest from the uploaded file and compares its canonical representation with the one stored. Any non-determinism or even a single whitespace misalignment in serialisation would cause verification failures.

To achieve determinism requirements, canonicalisation was applied before serialisation. The manifest lists fields, which are sorted according to a defined key order, and variable-length collections are normalised to a canonical form before encoding. CBOR (Concise Binary Object Representation, RFC 8949 [10]) was selected for manifest serialisation because its deterministic encoding mode defines a total ordering for map keys and a canonical representation for all data types, guaranteeing that the same manifest always produces the same sequence of bytes. Besides that, CBOR also offers a smaller binary encoding than JSON, making it more relevant in large volumes of stored manifests, and their sizes are included as an artefact metric in the benchmark.

3.5 Microservice Architecture Decision

A monolithic architecture cannot support stage-level latency attribution - a single-process binary cannot isolate the timing of its own internal phases from the measurement - and does not resemble the distributed service topologies used in production cloud environments. The system was therefore decomposed into three independently deployable microservices:

API Gateway. Designed to act as an interface that connects backend services and frontend input. Provides trust boundaries and enforces authentication. Routes and rate-limits requests to the corresponding downstream services. Client-side timings are measured at this stage.

Hasher Service. Ingests a file and computes a hash string using the algorithm specified by ID. Returns the computed digest. Exposes a dedicated endpoint for measuring service latency.

Manifest Builder Service. Composes the canonical manifest and immediately generates a signature to eliminate raw byte transfer between services. Stores the manifest in PostgreSQL and sends the signed object to MinIO. Verification requests are also handled here by accessing stored manifests and the signing pipeline needed to re-derive manifests for comparison.

Services occupy their dedicated Docker containers. Nginx [46] works as a reverse proxy in front of the API gateway, and Docker Compose [13] orchestrates the stack. From an architectural perspective, the decision to implement this prototype as microservices was not purely a matter of convenience. Telemetry at the stage level is only possible with separate microservices, and this is vital for evaluation. Each service performs its own computations and reports the latency of its own operations. This applies directly to RQ1, which helps determine which stage contributes most to PQC overhead, and to RQ3, which helps develop a benchmarking harness that produces reproducible measurements at a fine-grained level.

3.6 User Interface

Within the scope of this project, a web client serves solely as an illustrative navigation interface for demonstration and functionality testing. Browser-based solutions would inherently be unfit for benchmarking. Interface layers that add uncontrolled latency to the observed round-trip would contaminate the timing data and conflate cryptographic cost with rendering overhead.

However, because a basic UI is required, several options for interacting with the system were considered.

A Terminal User Interface (TUI) was attempted due to its minimal, terminal-based approach. It provides a lightweight way to build a selection menu that could be comfortably navigated. Yet it was eventually dropped because it required more thorough orchestration and debugging of visual components. It is also more complex to maintain and is less user-friendly, specifically for non-technical leads.

On the other hand, React [41] web interface built with Vite [16] provides a browser-based solution. The React ecosystem is industry-leading and widely adopted for a range of solutions. Thus, it was selected as the primary frontend tool. The web UI was therefore built to configure sign and verify flows and to inspect results. It serves only as a demonstration and is not used for benchmarking because its performance is irrelevant to the measurement results.

Benchmarking required a lean, low-overhead harness to automate performance measurement. This requirement prompted the building of a dedicated command-line interface (CLI) tool. Browser-based testing was explicitly ruled

out because it introduces variable latency from the browser’s network stack, outside the control of the testing environment, and would contaminate timing measurements. The CLI tool sends HTTP requests to the API gateway, measures latency using monotonic Rust Instant [77] timers, and produces a sequence of JSON records that can be deterministically replayed using a seed parameter. This also fits well into scripting automated campaigns.

3.7 Verification Service Design

Since integrity checks limited to hash matching are insufficient as a security control, a more sophisticated framework was designed. The verification service was developed to serve as a ”verification by re-execution” workflow. An attacker who can forge a manifest in the database could replace the stored signature with one for a different payload. Such tampering of the stored hash invalidates non-repudiation guarantees, so detecting this class of attack requires full cryptographic re-derivation. This means that the submitted file undergoes full re-execution: re-hashed, re-signed, and the resulting manifest is compared against the stored one. This design ensures that any substitution, no matter which component is involved, is surfaced as a verification failure rather than a silent pass.

The user submits the file to be checked and the request ID of the original signing operation. The service fetches the stored manifest, re-derives the manifest for the submitted file, and evaluates requested verification checks.

The provided hash match check verifies that the re-derived hash matches the hash stored in the manifest. The object hash match check verifies that the object stored in MinIO hashes to the same digest as recorded.

One signature validity check is then executed per active scheme to confirm that each stored signature correctly verifies against the canonical signing bytes. Then, manifest field consistency checks finally confirm if the parameters match between the re-derived and stored manifests.

Checks return a passed flag and a details string. The overall result is PASS if and only if all checks pass; otherwise, it is FAIL. More checks were added after each iteration to enhance the verification diversity. Earlier versions of the verifier simply checked that the hash matched, whereas signature and manifest field checks were added later, when it was found that a hash collision would not catch all classes of manifest substitution.

To build the end-to-end workflow, measurement scenarios were defined as a requirement to allow splitting it into separately measured stages.

The workflow scenario captures the complete sequence of uploading the file, hashing it, signing it, storing it in MinIO, persisting the manifest to PostgreSQL, and verifying.

The `sign_only` scenario captures the upload and signed-manifest generation path without any verification step.

The verification scenarios then isolate progressively narrower checks. `verify_manifest` compares manifest metadata only, without fetching the stored object from MinIO.

`verify_stored` verifies the hash of the stored MinIO object against the manifest record.

`verify_uploaded` re-hashes and re-signs the uploaded file, and compares the resulting manifest against one in the storage.

`verify_full`, as the name implies, combines stored-object hash verification, uploaded content re-signing, and manifest checking.

The scenario-level cost can be separated and compared by running each scenario as an independent condition in the benchmark matrix.

3.8 Security Hardening and Threat Modelling

Before benchmarking, the prototype was reviewed against common vulnerability classes in cloud-based file-handling systems using a STRIDE threat model [72] (spoofing, tampering, repudiation, information disclosure, denial of service, elevation of privilege). These classes of risk are relevant to a file upload and signing service:

Time-of-Check-Time-of-Use (TOCTOU) [9] is a vulnerability that may occur when a symlinked path could be swapped between the validation step and the file-read step, causing the service to process a file it did not intend to. This vulnerability was mitigated by checking each part of the file within the allowed upload directory using `symlink_metadata` at each component. Any path that contains a symbolic link is therefore rejected. Symlink following, if ever required, is gated behind a deliberate double-opt-in configuration mechanism designed to prevent accidental activation in deployments where it is not intended.

Null-byte path injection may occur when filenames containing a null byte (`\0`) are truncated at the OS level. This causes the service attempt to access an alternative destination. Paths containing a null byte are therefore rejected before passing them further.

Directory traversal may be exploited when path components containing the traversal sequence `../` escape the directory and access arbitrary filesystem locations. Component-level checks reject any directory that attempts to traverse above the root of the allowed ones. Therefore, a whitelist (defaulting to `/data/uploads`) is defined to enforce that all access is confined to the upload area.

The fourth is sensitive file exposure. Unrestricted path access could expose system files under prefixes such as `/etc`, `/root`, `/proc`, and `/sys`. To build a defence-in-depth security model, an 11-entry denylist of sensitive paths is enforced alongside the whitelist.

Beyond these, the prototype also underwent additional hardening. The API key comparison uses a constant-time equality check [31], so attackers cannot guess valid keys by timing the comparison. A role-based access control (RBAC) [68] model allows operations only by role (Admin > Operator > ReadOnly), preventing restricted operations on keys with limited privileges. Request rate is controlled using a token bucket algorithm [78] tied to the client’s IP address and User-Agent, limiting request frequency to prevent abuse. Internal service-to-service authentication requires tokens at least 32 characters long.

An audit history logs security-relevant events. Records include essential information about the event that would enable the reconstruction of the trail. API keys and signature bytes, along with other sensitive strings, are masked using a function that only reveals the first two and last components of a path. The intermediate parts are replaced with asterisks `***`. Failed or denied events are emitted at warning severity to make them visible in aggregation pipelines.

This phase of security hardening is far from ancillary; rather, it is a *sine qua non* condition for the results reported in this thesis. A prototype with exploitable path-traversal or authentication vulnerabilities would not model a realistic deployment; its latency measurements would reflect an attack surface rather than a production system, undermining any inference drawn about migration cost. By implementing the security controls described above before starting the benchmarking campaign, the prototype ensures that the timing data are captured in the conditions of a deployable service. This addresses RQ4 head-on: this implementation shows that integrating PQC signatures into a microservice architecture hardened according to the security best practices does not require any trade-offs in the security model. Indeed, the role-based access control and audit logging features demonstrate that the cryptographic non-repudiation guarantees provided by PQC signatures are mirrored in the operational layer of accountability, a feature particularly important in regulated cloud environments that require an audit trail.

3.9 Algorithm Selection

Algorithm selection and benchmarking are carried out in two phases. This is essential to minimise the processed data and extract only the insights needed for the most suitable algorithm combinations.

3.9.1 Phase 1: Baseline Establishment

This phase covers the entire signature list across all file sizes and scenarios, with the objective of establishing baselines and identifying the best-in-class representative performance profiles within each family.

Classical (4 profiles). RSA-PSS, ECDSA-P256, EdDSA, and HMAC-SHA256. These will serve as baselines against which the overhead of PQC is measured. Given the widespread use of the RSA-PSS algorithm as a signing technology in current cloud infrastructure, RSA-PSS serves as the natural baseline against which other signing overhead measurements are compared in Section 4. For the symmetric case, HMAC-SHA256 [32] offers the minimum signing overhead but lacks public verifiability and therefore is not a migration candidate.

Post-quantum (3 profiles). ML-DSA (FIPS 204 [55]), SLH-DSA (FIPS 205 [56]), and Falcon (standardised as FN-DSA in FIPS 206 [57]). These three algorithms represent NIST’s final selection of post-quantum digital signature algorithms. ML-DSA (Module-Lattice-based Digital Signature Algorithm) descends from the CRYSTALS-Dilithium submission; SLH-DSA (Stateless Hash-based Digital Signature Algorithm) descends from SPHINCS+; and FN-DSA (Falcon’s NIST standardised form) is based on NTRU lattices. They will henceforth be referred to in this dissertation using their official FIPS designations.

3.9.2 Hash Algorithms

The selection of hashing algorithms needed to be sufficiently widely adopted yet also represent candidates with distinct core cryptographic approaches. Therefore, SHA-256 [53] was selected as the prevalent, industry-standard baseline, BLAKE3 [47] was chosen as an emerging, modern, high-throughput alternative, and Keccak-256 (SHA-3 family [45]) as a structurally different hash candidate that shares no shared lineage with SHA-2. This decision covers a range of security bases and throughput profiles, allowing the separation of the latency contribution of hashing from that of signature generation.

3.9.3 Phase 2: Hybrid Benchmarking

The two best-performing schemes from each category in Phase 1 were identified and carried forward into the Phase 2 hybrid benchmarking campaign. In this configuration, both a classical and a post-quantum signature are computed in parallel and independently. The resulting fingerprints are stored in the manifest envelope, and verification requires both to pass. This approach presents a dual-signing strategy in accordance with the NIST transition

guide [58]. As per their report, an organisation seeking to deploy quantum-safe protection can maintain classical interoperability. Running the campaign in two phases, therefore, places the more expensive hybrid measurements onto the most practically promising configurations.

3.10 Benchmarking Framework Design

3.10.1 Experimental design

The experiment follows a full-factorial design with these variables:

- signature profile (7 levels)
- hash algorithm (3 levels)
- payload size (5 levels)
- workflow scenario (6 levels)

The campaign measures client-observed and server-side latencies, calculated throughput (MiB/s), and artefact sizes (bytes), all derived independently. Controlled variables config must include storage state and inter-run delay (350 ms). Randomisation via a seeded PRNG and a warm-up protocol of 10 runs per condition control for order effects and cold-start bias, respectively.

The benchmark CLI constructs a condition matrix: every combination of (signature profile, hash algorithm, file size, bucket, scenario) constitutes a condition. Each condition is run multiple times, and the resulting per-run records form the dataset for statistical analysis. In Phase 1, the matrix spans all seven profiles and all three hash algorithms. The analysis script then identifies the two best-performing representatives in each category and the two best-performing hash algorithms, forming the input set for the Phase 2 hybrid matrix. The multi-phase technique supports the exploratory breadth of Phase 1 whilst directing a deeper measurement effort to Phase 2. This allows for a comprehensive comparison of only those configurations most likely to be operationally relevant.

3.10.2 Warm-up runs

This setup configures 10 warm-up runs per condition to be executed before any measured run begins. Warm-up runs are marked and excluded from subsequent measurement aggregation and analysis. The rationale behind such runs is to allow the OS cache to load input files and flush latency from cold runs for the first requests to services. Without such an exclusion, cold-start effects could artificially inflate the median with relatively few runs and systematically bias results across different conditions.

3.10.3 Measured runs

One hundred measured runs are executed per condition. The number of runs was chosen so that the width of a 95% bootstrap confidence interval would be meaningful while the total campaign duration remained manageable.

3.10.4 Seeded randomisation

The ordering of conditions within each round is randomised using a seeded `StdRng` [76] (a deterministic pseudorandom number generator). All runs using the same seed value reproduce the exact same condition ordering, enabling independent replication of any campaign. If no seed is specified, a random 64-bit value is sampled and recorded in the campaign metadata, so the ordering is always reproducible after the fact. This addresses RQ3, ensuring that the ordering of the runs is random, so the results are not confounded by order-dependent artefacts, but also deterministic, allowing exact reproduction.

3.10.5 Inter-run delay

A 350 ms pause is inserted between consecutive runs. This delay gives the OS scheduler time to complete any pending I/O, allows the database connection pool to recycle its connections, and reduces the impact of OS scheduling jitter between runs. The delay is configurable, but 350 ms was found to be the minimum necessary to keep the relative IQR within the quality gate threshold across most conditions.

3.10.6 Storage state modes

The system supports warm and cold storage state modes. Cold mode generates a fresh dataset for each campaign run using new random files, guaranteeing that no prior content is cached in the OS or MinIO's storage. Warm mode targets a pre-generated dataset measurement. This ensures that subsequent accesses benefit from the stored cache and the connection pool's warm-up. For Phase 1 of the campaign in Section 4, warm mode was used to separate cryptographic algorithm costs from the cold-start I/O variance. The same campaign tooling supports cold mode as an extended condition.

3.10.7 Campaign orchestration

To ensure reproducibility, the execution sequence is codified in an orchestration script that manages the full configuration and lifecycle. The script also supports resume-on-failure so that an interrupted campaign restarts from the

last completed condition rather than from scratch. It is a useful quality-of-life feature, critical for long campaign runs where a mid-run failure would otherwise discard hours of completed work. A smoke-test mode with reduced runs allows quick validation of the full pipeline before committing to a full campaign. Together, these make the benchmarking procedure more bearable and fully repeatable from a single command.

3.11 Dataset Design

The benchmark dataset was designed to assess efficiency across a wide range of file sizes. Buckets of sizes 10 KB, 100 KB, 1 MB, 10 MB, and 50 MB were defined, as these boundaries reflect the range of file sizes commonly encountered in backend cloud workflows - from small configuration files and manifests to large binary assets.

Similarly, several file types were chosen for variance: `.bin` (random binary content), `.txt` (random printable ASCII text), `.json` (structured key-value records), `.csv` (tabular data rows), and `.md` (Markdown-formatted prose). Types are assigned in round-robin order within each bucket, ensuring an even distribution across all types at every size. The presence of several file types ensures coverage of any compression or padding effects arising from the file's content structure.

32 files are generated per bucket (160 files total). Randomness of the generation is determined by SHA-256 hash chaining from a fixed seed string (`pqc-hons-dataset-v2`). This is a step further from the operating system's random number generator, as each file's byte content is deterministically derived by repeatedly hashing a seed-prefixed counter. Thus, from this single seed string alone, one may regenerate a byte-for-byte identical dataset regardless of platform or when it was generated. This ensures any benchmarking lab can recreate the identical dataset from the repo, facilitating independent replication.

The dataset source is recorded to document the metadata and generation parameters. This is stored alongside the benchmark output for the full layout. These records allow any benchmark run to be linked to its exact input file and ensure that an independent laboratory can regenerate a byte-for-byte identical dataset and reproduce the campaign without any coordination beyond cloning the repository.

3.12 Telemetry and Metrics Collection

A fundamental challenge in system-level cryptographic benchmarking is that a single end-to-end timer conflates the cost of the cryptographic primitive

with the shared infrastructure floor - the HTTP round-trips, database writes, and object-store operations that are identical across all conditions. Without finer-grained attribution, a difference in end-to-end latency between two signature schemes could originate from the primitive itself or from incidental I/O variation. To address this, the prototype records telemetry at two levels: client-observable and server-internal.

3.12.1 Client-side timings

The benchmark CLI implements the user-observable workflow using Instant timers that record the total workflow latency and per-phase durations. Throughput is derived as file size divided by elapsed time and reported in MiB/s. These metrics capture the end-to-end migration cost that a practitioner would observe when switching signature schemes - the answer to RQ1 at the system boundary.

3.12.2 Server-side timings

Microservices record the duration of their own atomic operations and embed them in the HTTP response, so that the CLI can log them along with its own measurements. This enables stage-level attribution, and per-algorithm signing fields isolate the cryptographic primitive itself. This attribution capability is not present in prior microbenchmark studies, but it reveals observed latency differences and whether they originate from the cryptographic algorithm or the surrounding infrastructure. Such a setup allows one to draw conclusions about PQC costs from direct evidence rather than from end-to-end totals.

3.12.3 Artefact metrics

Beyond latency, the telemetry records signature sizes and manifest component sizes for every run. Metrics also capture derived overhead ratios for signature bytes, expressed as a percentage of file size, and total storage amplification. This provides the storage-cost dimension that is invisible in latency-only benchmarks but shows up at cloud billing scale. Together, this directly answers RQ1's storage sub-question.

These three layers allow RQ1 to be answered from different angles. Client-side totals show the full cost one would see in practice; server-side timings identify which parts of the system drive that cost; artefact metrics reveal how storage costs increase as the amount of stored files increases.

3.13 Statistical Analysis Methodology

Latency measurements in such systems are right-skewed, with heavy tails due to OS scheduling interruptions. The mean and standard deviation in parametric statistics are sensitive to these events and would misrepresent typical performance. This section describes the statistical framework used to produce comparable estimates across different conditions, with wide latency ranges.

3.13.1 Central tendency

The median (P50) is used as the primary measure of central tendency in preference to the mean, precisely because it is robust to the heavy-tailed outliers described above. It provides a representative picture of typical performance under normal workloads, without being inflated by rare scheduling spikes.

3.13.2 Dispersion

The interquartile range ($IQR = P75 - P25$) is a dispersion measure as it is not sensitive to extreme values. The relative IQR adjusts for scale, allowing comparison of how consistent the results are despite different baselines. For example, it enables a fair comparison between SLH-DSA at 1,600 ms and a quicker HMAC at 0.20 ms.

3.13.3 Confidence intervals

95% bootstrap confidence intervals [14, 15] are computed for the median of each condition using 2 000 resamples with replacement. Bootstrap CI estimation makes no distributional assumptions, which are appropriate for latency data that may be right-skewed. It remains valid regardless of the absolute scale of the measurement, which is essential when comparing conditions with such a large difference. A minimum of 3 samples is required before a CI can be computed; otherwise, the conditions are flagged as insufficient.

3.13.4 Ratio analysis

To directly count PQC overhead relative to the classical, the ratio of each profile’s median to the RSA-PSS median is computed for each condition. Uncertainty is propagated by resampling both the numerator and denominator together, yielding a bootstrapped 95% CI for the overhead ratio. This produces quantified, confidence-bounded comparisons of the form ”ML-DSA imposes

$X \times$ the end-to-end latency of RSA-PSS (95% CI $[Y, Z]$)”, which addresses RQ1.

3.13.5 Quality gates

The data entry must satisfy quality criteria before committing to evaluation. This feature is designed to exclude unstable or partial measurements.

The condition success rate must be at least 90%, so that failed runs cannot dominate the outcome and provide no value to the analysis. The same applies to the relative IQR, which must not exceed 50% of the median, and unstable conditions must be excluded from the analysis. Additionally, server metric coverage must ensure that every successful run has the full server-side telemetry required. To guarantee consistent results, at least 3 measurements are required before a condition can proceed to the summaries.

Conditions failing any gate are labelled with the failure reason and rejected from plots and ratio tables. This framework ensures that noisy or incomplete data cannot distort the conclusions.

The analytics pipeline computes quality results for each condition. This includes tables of confidence intervals and ratio summaries, which are used in the experimental analysis presented in Section 4.

4 Experimental Results and Analysis

This section presents and discusses the results of the formal Phase 1 benchmarking campaign. The section is organised as follows: Section 4.1 describes the provenance and quality of the dataset; Sections 4.2-4.6 report the measured performance of each dimension; Sections 4.7-4.12 analyse these findings across multiple dimensions; and Section 4.13 derives the algorithm selection for the Phase 2 hybrid campaign. Measurements presented are medians across 100 warm-up runs per condition. Server timings serve as the basis for conclusions about the cryptographic stage, as required by the procedure (Section 3); client timings include the full HTTP round-trip and are used for full workflow comparisons.

4.1 Campaign Provenance and Quality

Phase 1 of the campaign was launched against a warm, pre-existing dataset (see Section 3.11), with the configuration fixed before the run began. Such a measure is designed to prevent post-hoc adjustment. The campaign identifier is recorded in the output artefacts to link the measurement to its context. Table 2 summarises the campaign configuration.

Table 2: Formal Phase 1 campaign summary.

Parameter	Value
Campaign label	formal-1
Storage state	Warm
Repeat index	1
Signature profiles	RSA-PSS, ECDSA, EdDSA, HMAC, ML-DSA, SLH-DSA, FN-DSA
Hash algorithms	SHA-256, BLAKE3, Keccak-256
Payload buckets	10 KB, 100 KB, 1 MB, 10 MB, 50 MB
Scenarios	6 (workflow , sign_only , and four verify variants)
Total conditions	630
Warm-up runs per condition	10
Measured runs per condition	100
Total measured runs	63 000
Random seed	43
Bootstrap samples	2 000
Host CPU	AMD Ryzen 7 6800H, 16 logical cores
Total memory	31.4 GiB
Operating system	Linux (x86_64)
Build profile	Release

A quality check of the results confirmed the dataset’s validity before evaluating 63 000 measured runs, where 62 999 completed successfully (99.998% success rate). The one failing run did not belong to a profile or bucket, indicating isolated interference, which is negligible in this run. 630 conditions passed every quality gate and therefore is valid for further analysis. The median coefficient of variation (CV) for the client was 6-7.5% in all non-SLH-DSA conditions, and below 1% for SLH-DSA.

4.2 Hash Algorithm Performance

The hash stage is the operation where the cost scales with the size. This is because this is the primary operation which ingests variable payload dimensions. Subsequent signing operations operate on a fixed-length digest, whose size depends on the hashing algorithm, regardless of the original file size. Hash performance, therefore, determines the extent to which total workflow latency increases with payload and distinguishes the three candidate algorithms at large file sizes.

Table 3 reports server-side hash latency (`server_hash_ms`) and derived throughput for the `sign_only` scenario. Crucially, signature profiles in this prototype share the same hash stage. Decoupling the hash measurement isolates the profile-independent throughput scaling, proving that limits at massive payloads are not solely determined by asymmetric cryptography but rather by the hash function (file size divided by hash latency).

Table 3: Hash stage latency and throughput (`sign_only` scenario, warm, median).

Hash	Latency (ms)					Throughput at 50 MB (MiB/s)
	10 KB	100 KB	1 MB	10 MB	50 MB	
SHA-256	0.16	0.53	3.74	37.5	182	274
BLAKE3	0.16	0.53	3.61	35.6	170	294
Keccak-256	0.17	0.68	5.71	56.4	280	179

SHA-256 and BLAKE3 track closely at small sizes (both ≈ 0.16 ms at 10 KB) before BLAKE3 opens a small but consistent advantage at large payloads: at 50 MB BLAKE3 achieves 294 MiB/s versus 274 MiB/s for SHA-256, a 7.3% improvement. SHA-256 on this platform benefits from Intel SHA-NI hardware acceleration [24]; on ARM-based cloud instances (AWS Graviton [6], Google Axion [21]) with ARMv8 cryptography extensions [7], the gap narrows further.

Keccak-256 is consistently slower across all payload sizes, reaching only 179 MiB/s at 50 MB - a 39% deficit relative to BLAKE3 and 35% relative to SHA-256. This is mainly because Keccak only runs in software, unlike SHA-256 and, on newer processors, BLAKE3, which both enjoy hardware acceleration for the Keccak permutation on server-class processors. The absolute difference is modest at small sizes (e.g. only 0.15 ms longer than BLAKE3 at 100 KB) but grows to 110 ms at 50 MB, where it accounts for approximately 12% of the total end-to-end latency for non-SLH-DSA profiles. In practical terms, this means that the choice of hash algorithm is a second-order concern for small-object workloads but becomes material for large-payload pipelines - a pattern that recurs in the end-to-end analysis of Section 4.6.

4.3 Signing Performance

Table 4 reports the server-side signing cost per algorithm for the `sign_only` scenario with SHA-256 as the hash. Since the hash stage is identical across

all rows, this lets crypto cost to be isolated. Sign latency is triggered solely by the resulting 32-byte digest, therefore, profiles remain flat and are not dependant on the payloads. This confirms that cryptographic processing does not degrade during larger object transfers over the network. The minimal variance across bucket sizes merely reflects temporary run-to-run jitter in the system.

Table 4: Server-side signing latency by profile (SHA-256 hash, `sign_only` scenario, warm, median ms).

Profile	Family	10 KB	100 KB	1 MB	10 MB	50 MB
HMAC-SHA256	Classical	0.21	0.19	0.18	0.23	0.28
EdDSA	Classical	0.25	0.22	0.23	0.28	0.32
FN-DSA	PQC	0.42	0.38	0.38	0.51	0.54
ECDSA	Classical	0.50	0.47	0.47	0.61	0.64
ML-DSA	PQC	0.79	0.81	0.74	0.81	0.97
RSA-PSS	Classical	4.73	4.68	4.79	5.07	5.26
SLH-DSA	PQC	1595	1595	1597	1601	1605

Several findings merit discussion.

4.3.1 Falcon (FN-DSA) outperforms ECDSA

FN-DSA signs in approximately 0.4 ms - faster than ECDSA (0.5 ms) and only $1.7\times$ slower than EdDSA. This is the main practical surprise of the dataset. In most standard textbook treatments of PQC, signature schemes are said to incur a very large relative performance penalty compared to schemes from the ECC family. Here, thanks to NTT-based Gaussian trapdoor sampling [57], FN-DSA offers very competitive sign latency on current server hardware. The gap between FN-DSA and EdDSA (≈ 0.17 ms) is smaller than the gap between RSA-PSS and EdDSA (≈ 4.5 ms).

4.3.2 ML-DSA is in the same order of magnitude as ECDSA

ML-DSA signs in 0.74-0.97 ms across bucket sizes, approximately $1.5\text{-}2\times$ the cost of ECDSA and $3\times$ the cost of EdDSA. For a service executing fewer than a few thousand sign operations per second, this difference (<0.5 ms) is unlikely to constitute a throughput bottleneck.

4.3.3 RSA-PSS is by far the slowest classical signing algorithm

RSA-PSS takes between 4.7-5.3 ms, which is approximately $20\times$ that of EdDSA and $10\times$ that of FN-DSA. This cost is inherent to the RSA modular exponentiation operation and is well-understood. It means that any service currently using RSA-PSS can begin planning a migration strategy to a quantum-resistant algorithm and achieve a substantial improvement in signing speed by switching to FN-DSA or ML-DSA.

4.3.4 SLH-DSA is unusable for online workloads

SLH-DSA requires approximately 1 600 ms to sign, irrespective of payload size. This is a consequence of its hash-based construction: SLH-DSA generates many chained Merkle tree layers during signing, each requiring multiple rounds of the underlying hash function [56]. A service signing at even one operation per second would need to dedicate a full core exclusively to SLH-DSA signing, and its client-observable round-trip would be dominated by signing latency rather than network overhead. SLH-DSA is therefore not viable for any latency-sensitive cloud signing path.

4.4 Verification Performance

Verification asymmetry - the ratio of signing to verification cost - is operationally significant in cloud workloads where items are signed once and verified many times. Table 5 reports server-side verification latency from the `verify_manifest` scenario, isolating the cryptographic verification primitive from general storage I/O. As visibly demonstrated, the verification values are incredibly stable across payload sizes (represented by the 10 KB and 10 MB anchors). This systemic payload independence is highly beneficial for read-heavy architectures that rely on predictable bulk verification routing.

Table 5: Server-side verification latency by profile (SHA-256 hash, `verify_manifest` scenario, warm, median ms).

Profile	Family	10 KB	10 MB
HMAC-SHA256	Classical	0.20	0.20
FN-DSA	PQC	0.25	0.25
EddSA	Classical	0.26	0.26
ML-DSA	PQC	0.39	0.39
ECDSA	Classical	0.54	0.51
RSA-PSS	Classical	0.68	0.67
SLH-DSA	PQC	1.61	1.62

The rank order differs markedly from signing. FN-DSA achieves the fastest public-key verification in the entire suite (0.25 ms), beating EddSA by 0.01 ms and ECDSA and RSA-PSS by multiples of 2 and 2.7, respectively. This is consistent with the design of the Falcon lattice trapdoor, as verification requires only a matrix-vector product, which is inexpensive compared to the operations required for signing [57].

ML-DSA verification (0.39 ms) is faster than ECDSA and RSA-PSS verification, reversing their relative signing costs. For a read-heavy service that verifies far more often than it signs, both FN-DSA and ML-DSA offer a good trade-off. Both offer competitive or superior verification speed, with verification latency no worse than that of the classical alternatives.

SLH-DSA verification (1.61 ms) is orders of magnitude faster than its signing cost ($\approx 1\,600$ ms). This is what is expected from hash-based signatures: signing is costly, but verification requires only evaluating hashes along a fixed authentication path. This asymmetry makes SLH-DSA viable for some specific use cases (see Section 4.12) but not as a general-purpose cloud signing algorithm.

These asymmetry results have a direct operational implication for RQ1. For cloud workloads where read operations dominate, objects are verified far more often than they are signed. Therefore, FN-DSA and ML-DSA offer a more favourable profile than signing-focused metrics would suggest.

4.5 Artifact Overhead

Signature schemes impose a storage cost through the size of the signature bytes stored in the manifest. In cloud object storage environments, manifests for every signed object must be stored, transmitted, and (during verification) read back, which adds extra overhead and translates into higher costs. At

the scale of millions of objects, the byte count adds up to appreciable storage and egress charges.

Table 6 presents the signature size chart for each profile and the overhead as a percentage of the file size for the bucket parameters. This storage amplification must be framed as a considerable design threat rather than a secondary byproduct, as it directly affects otherwise viable post-quantum transitions deployed at massive cloud scales.

Table 6: Signature sizes and overhead (SHA-256, `sign_only` scenario, warm, median).

Profile	Family	Sig. bytes	% of 10 KB	% of 1 MB
HMAC-SHA256	Classical	30	0.29%	0.003%
EdDSA	Classical	63	0.62%	0.006%
ECDSA	Classical	69	0.67%	0.007%
RSA-PSS	Classical	384	3.75%	0.037%
FN-DSA	PQC	654	6.39%	0.062%
ML-DSA	PQC	3 309	32.3%	0.315%
SLH-DSA	PQC	7 854	76.7%	0.749%

For objects of 1 MB or more, the overhead of all schemes is negligible (all below 1%). The differentiating parameter is small files, which make up the majority of objects in such deployments. Configuration files and logs are the most common and range from 1 to 100 KB.

At 10 KB, ML-DSA imposes a 32% overhead: every signed 10 KB object requires approximately 13.3 KB of storage, yielding a storage amplification factor of $1.32\times$. SLH-DSA is worse still at 77%, equivalent to storing each 10 KB object in approximately 17.9 KB. FN-DSA, by contrast, adds only 6.4% at 10 KB - smaller than RSA-PSS's 3.75% by less than a factor of two, and an order of magnitude smaller than ML-DSA.

To better showcase the financial implications at production scale, consider a hypothetical automated log system generating around 1 billion 10 KB records per month. Retaining manifests signed with ML-DSA would add approximately 3.3 TB of storage overhead each month compared with classical ECDSA signatures. Over a five-year period, this pipeline single-handedly would accumulate nearly 200 TB of additional data due to larger signature sizes, assuming raw S3 Standard storage. In a cloud environment such as AWS S3, this translates to much higher monthly storage and outbound data costs. This projection is simply explanatory - the actual overhead depends on the configuration. But it fundamentally emphasises a critical point where selecting

a signature algorithm is not limited to latency or security level measurement. Artefact size is an important attribute of the migration planning because a scheme that appears acceptable on latency grounds may impose unacceptable storage costs at cloud-scale billing, a trade-off that latency-only evaluations essentially miss.

The storage increase ratio across buckets shows the scale of payload bloat introduced by the storage capabilities. The sheer size of signatures adds bandwidth penalties on schemes like ML-DSA, or particularly SLH-DSA. At the same time, FN-DSA achieves a remarkably close profile to classical results.

4.6 End-to-End Latency

The **workflow** scenario provides measurements the most relevant to the operations. The full round-trip travels from file upload through each service as observed by the client. This is the latency that a consumer experiences.

Table 7 reports client-observed end-to-end latency for the **workflow** scenario with SHA-256, encapsulating all external HTTP and internal database processing events along the pipeline.

Table 7: Client-observed end-to-end workflow latency (SHA-256, **workflow** scenario, warm, median ms).

Profile	Family	10 KB	100 KB	1 MB	10 MB	50 MB
HMAC-SHA256	Classical	31.3	32.2	48.1	208	908
EdDSA	Classical	31.5	32.5	48.8	206	925
FN-DSA	PQC	31.8	32.8	48.7	211	896
ECDSA	Classical	32.1	33.5	49.0	211	913
ML-DSA	PQC	32.8	33.7	49.8	210	920
RSA-PSS	Classical	37.4	38.6	54.4	215	927
SLH-DSA	PQC	1 633	1 633	1 652	1 813	2 522

Building on this, Table 8 presents each profile’s latency as a ratio relative to the RSA-PSS baseline, along with bootstrapped 95% confidence intervals. As an empirical migration marker, ratio values scaling below 1.0 fundamentally indicate performance faster than the historic RSA-PSS standard, achieving clear parity for modernisation.

Table 8: Ratio of each profile’s end-to-end median to RSA-PSS median (SHA-256, `workflow` scenario, warm).

Profile	Family	10 KB		50 MB	
		Ratio	95% CI	Ratio	95% CI
HMAC-SHA256	Classical	0.837	[0.831, 0.842]	0.979	[0.950, 1.008]
EdDSA	Classical	0.841	[0.834, 0.846]	0.998	[0.964, 1.025]
FN-DSA	PQC	0.851	[0.842, 0.856]	0.966	[0.935, 0.997]
ECDSA	Classical	0.859	[0.851, 0.865]	0.985	[0.962, 1.016]
ML-DSA	PQC	0.878	[0.870, 0.883]	0.992	[0.970, 1.017]
SLH-DSA	PQC	43.66	[43.41, 43.80]	2.719	[2.673, 2.794]

The ratio table reveals two structural findings that directly address RQ1 (“What is the end-to-end performance impact of integrating PQC signing into a cloud file-handling workflow compared to classical schemes?”).

4.6.1 Finding 1: FN-DSA and ML-DSA are faster than RSA-PSS end-to-end

At 10 KB, FN-DSA achieves a ratio of 0.851 and ML-DSA 0.878 relative to RSA-PSS - both are faster than the classical baseline. At 50 MB, all non-SLH-DSA profiles converge within the 95% confidence interval of RSA-PSS (all CIs overlap with 1.0), meaning no statistically significant overhead is attributable to the signature algorithm at large file sizes. The dominant cost at 50 MB is the network upload and storage I/O, not cryptography.

4.6.2 Finding 2: The network and I/O floor dominate at all but the smallest files

A fixed overhead of approximately 15-18 ms is seen across all profiles at small file sizes, attributable to HTTP round-trips, database persistence, and MinIO object-store operations common to all conditions. This floor means that even a signature primitive costing 1 ms (as ML-DSA does) adds only 3-6% to the total round-trip for 10 KB objects. RSA-PSS’s 5 ms signing cost, by contrast, is a non-trivial fraction of this floor and is the primary reason it is consistently slower than all other profiles at small sizes.

4.6.3 Finding 3: SLH-DSA imposes prohibitive overhead for interactive use

SLH-DSA’s ratio at 10 KB is $43.7\times$ RSA-PSS (95% CI [43.4, 43.8]). Even at 50 MB, where signing accounts for only a small fraction of the upload latency, SLH-DSA remains $2.7\times$ slower than RSA-PSS due to its 1.6-second signing time exceeding the 50 MB upload time on localhost. No workflow involving interactive file signing can absorb this cost.

4.7 Cost of Hybrid vs Classical

To establish the viability of migration, the baseline performance penalty of introducing PQC algorithms must be determined. Figure 1 depicts this comparison by demonstrating the overhead introduced by hybrid configurations relative to purely classical baselines.

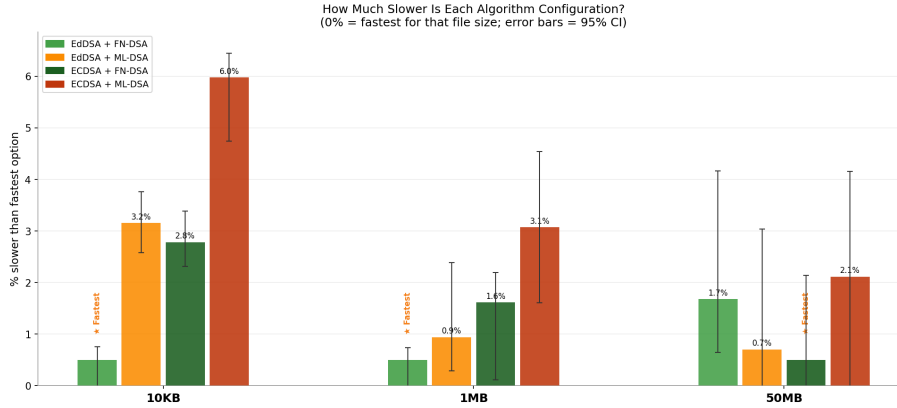


Figure 1: Hybrid overhead relative to the fastest configuration at each payload size.

The data challenges the belief that migrating to quantum-resistant schemes inherently entails severe performance penalties. As shown above, hybrid configurations incur less than a 6% overhead at 10 KB payloads, with the trend dropping to a statistically negligible 2.5% penalty at 50 MB loads. This indicates that network transmission and storage overhead completely dwarf the already low tax imposed by the signature process at standard cloud densities. The FN-DSA hybrid configuration, therefore, sets an end-to-end threshold comfortably within the classical boundaries. This means that providers can achieve quantum survivability without overprovisioning infrastructure, unless they rely on serial hashing constructions such as SLH-DSA.

4.8 Bottleneck Analysis

To understand where execution time is actually spent across the full workflow, Figure 2 decomposes server time to independent stages.

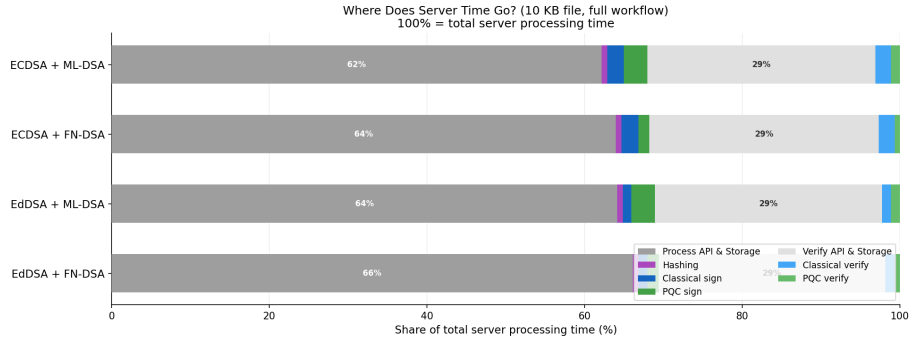


Figure 2: Server-side time decomposition for hybrid configurations (10 KB, full workflow).

The derived implications show that process API and storage operations account for 62-66% of total server time, proving that API handling and I/O are limited by throughput. The cryptographic operations required by the NIST standards do not constitute system limits. Thus, backend systems scaling for quantum resistance must optimise their database connection pools and routing access before focusing on latency.

To the contrary, beyond compute time, Figure 3 analyses storage inflation as a percentage of the file size.

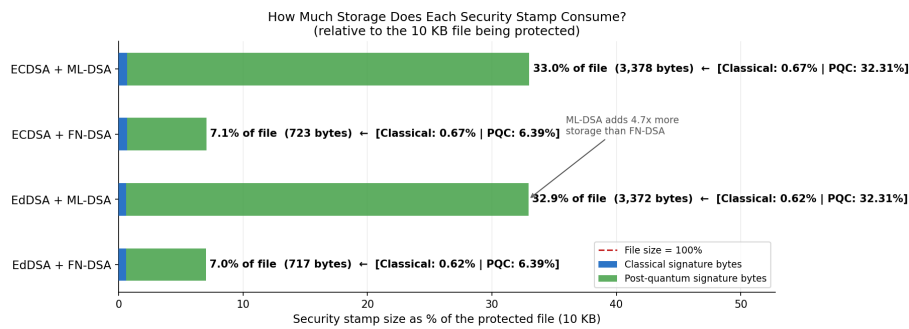


Figure 3: Hybrid signature size as a percentage of the protected 10 KB file.

The data reveal that ML-DSA-based pairings incur approximately 33% storage overhead for objects below 10 KB, whereas FN-DSA-based pairings incur only 7%. This payload congestion increase dictates network saturation

and storage inflation. It is not about execution-time compute limits; these limits will control the most restricted PQC transitions at commercial scale.

4.9 Algorithm Tradeoffs

Investigating algorithm trade-offs fundamentally requires dissecting verification imbalance. Cloud object stores remain mostly read-heavy systems where objects are signed once upon creation but then verified extensively throughout their lifecycles. Figure 4 highlights this property, displaying FN-DSA’s distinctive advantage in verification latency.

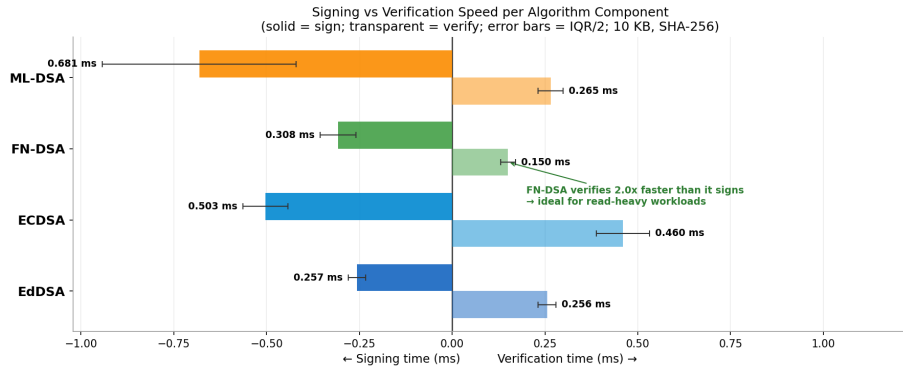


Figure 4: Signing vs. verification latency per algorithm (10 KB, SHA-256, IQR/2 error bars).

By verifying algorithms 2.0× faster than it signs them, FN-DSA presents itself as the ideal lattice-based primitive for read-heavy architectures. Optimising for signing bounds alone mischaracterises the true operational constraints in these pipelines, where verification throughput naturally dictates structural limits and endpoint responsiveness.

Furthermore, Figure 5 maps out how endpoint throughput scales differentially under high load across various algorithmic schemes.

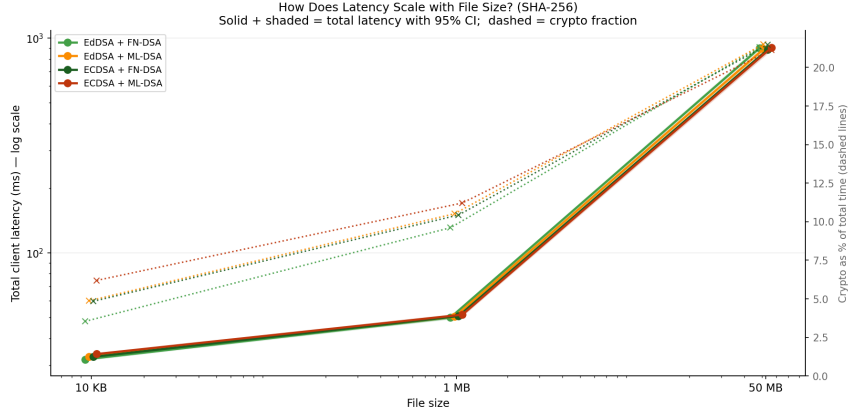


Figure 5: End-to-end throughput (MiB/s) by profile across payload sizes.

It is evident that all profiles (except excluded SLH-DSA) produce nearly identical throughput curves across varying payload sizes, demonstrating unified limits driven by common I/O interfaces. SLH-DSA, however, is separated by two to three orders of magnitude at smaller sizes, reaffirming that its hash-based design yields unacceptably low processing volume for synchronous object ingestion.

4.10 Primary Performance Analysis

Quantifying exactly how much PQC impacts a pipeline requires mapping its precise percentage contribution against established baselines. Figure 6 visualises PQC’s true contribution to end-to-end load times.

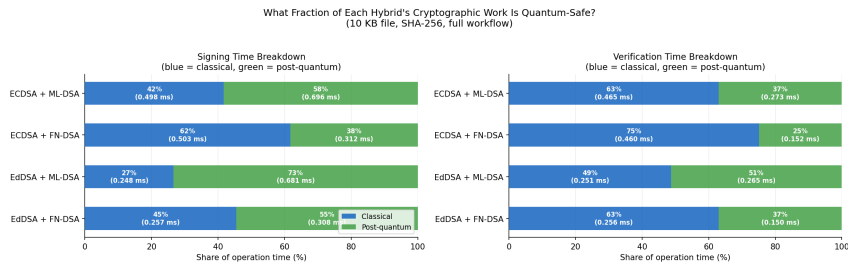


Figure 6: Quantum Signature contribution mapping out the true payload percentages.

These results give clear evidence that PQC overhead, as expected, is most pronounced for small payloads. However, FN-DSA maintains strict limits and remains highly competitive with RSA-PSS across the full analytical size

range. Conclusively, PQC is not a life-changing performance tax on basic latency mechanisms.

Figure 7 reinforces this by summarising the tangible temporal cost of transitioning a conventional architecture to a fully hybrid platform.

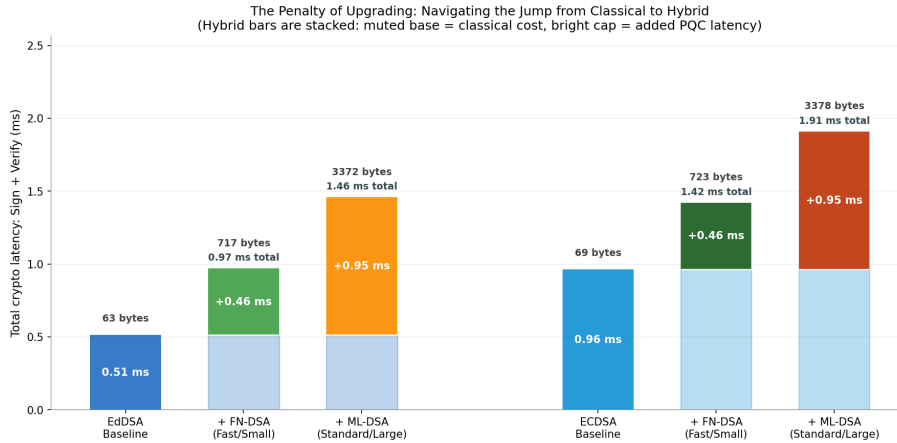


Figure 7: The cost of hybrid upgrade: classical baseline (muted) plus the added PQC latency (bright cap).

For the fundamental lattice-based schemes evaluated here, hybrid migration operates at algorithmic latencies well below historical RSA-PSS averages. The smallest and fastest combination, EdDSA + FN-DSA, guarantees that the overall transition can be absorbed dynamically. As a result, software transitions can definitely proceed without simultaneously requiring extensive physical infrastructure overprovisioning to counteract cypher compute spikes.

4.11 Infrastructure and Scalability

Determining what happens to these algorithmic constraints at a large operational scale is necessary before formulating broader recommendations. Using baseline metrics, we extrapolate throughput maxima imposed under intense operational limits. As seen in Figure 8, sequential transaction processing predictably drops under hybrid execution.

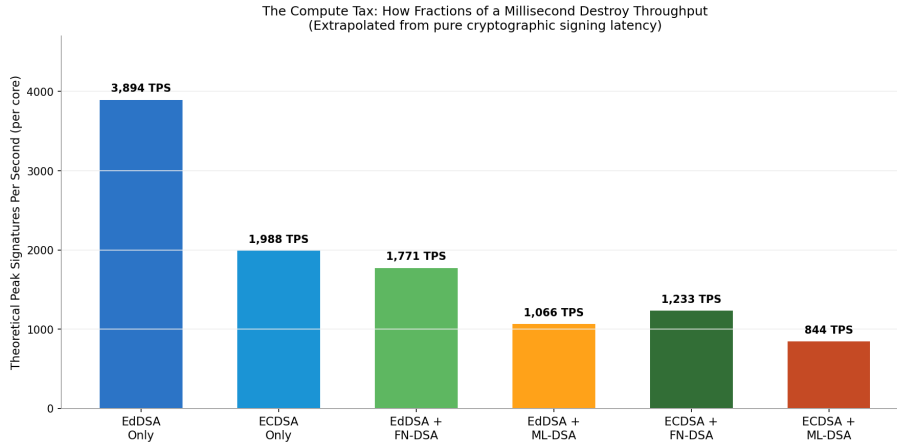


Figure 8: Theoretical peak signing throughput (signatures per second, single core) extrapolated from measured cryptographic latency.

Even given this constraint, EdDSA-only achieves a peak theoretical baseline (3,894 TPS), and the fastest hybrid (EdDSA + FN-DSA) still sustains 1,771 TPS. This means that a single lightweight processor core comfortably exceeds throughput expectations for almost any microservice. Algorithmic saturation alone is not enough to compromise most deployments.

Similarly, Figure 9 outlines the proportional bandwidth tax extrapolated across heavy, automated pipeline transmissions encompassing 1 million discrete requests.

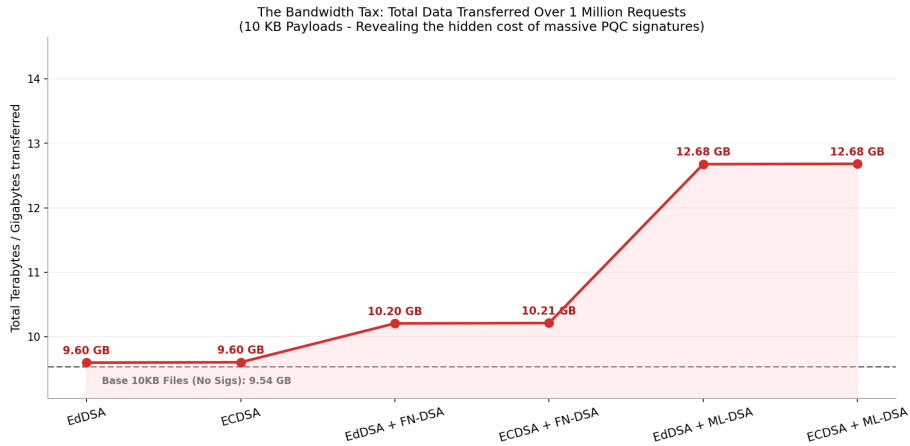


Figure 9: Bandwidth tax: total data transferred over 1 million 10 KB requests.

In scenarios where classical architectures introduce a negligible baseline padding (9.60 GB over scale), FN-DSA hybrids impose roughly a 6-7% transfer

growth, and ML-DSA inflates transmissions by upwards of 33%. As cloud solutions approach hard limits, this suggests that network sizing and cloud tariffs will impose strict upper bounds, likely not the internal cryptographic execution loops. Hence, organisations that plan to migrate must begin implementing smart data compaction frameworks to accommodate payload inflation.

4.12 Practical Scenarios

A thorough systemic examination culminates in concrete deployment heuristics mapped practically across distinctive infrastructure roles. Figure 10 acts as a heatmap establishing which configurations excel within typical cloud implementation contexts.

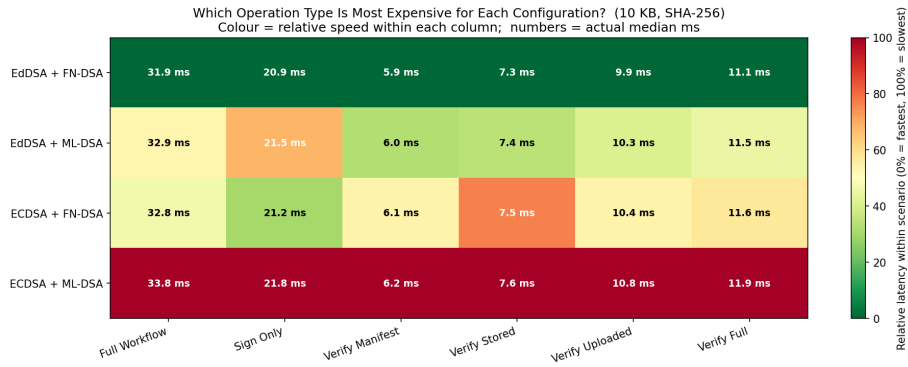


Figure 10: Scenario $\times$ configuration latency heatmap (10 KB, SHA-256).

Systems mandating complete, highly concurrent online operations consistently favour lightweight, low-overhead schemes that closely match the EdDSA and FN-DSA profile bounds. However, in contexts defined purely by long-horizon archival constraints - such as certificate authorities or deep-storage offline attestation systems - algorithmic speed becomes structurally irrelevant compared to mathematical certainty. Utilising conservative standards such as SLH-DSA in those circumstances ensures an absolute risk-aversion framework against unproven cryptographic derivations. A successful security posture thus dynamically allocates signature schemes strictly based on exact environmental constraints, rather than uniform policy.

4.13 Phase 2 Carry-Forward Selection

Phase 1 covered the performance of selected signature profiles and hash algorithms, during which a map of the design space was generated. Running

a full hybrid campaign over all possible combinations would expand the condition count to an unmanageable number. Moreover, most combinations would cover configurations already shown to be not as viable. Therefore, the Phase 2 carry-forward selection narrows the matrix to only the configurations most likely to represent practical migration paths. To align with current trends in cryptography strategy migration, there needs to be several selection criteria, importantly, not being bound to performance. Hence, Phase 2 campaign defines these critical attributes for choice:

The selection criteria were signing and verification performance, signature size and storage amplification, standardisation status, and viability for cloud-scale deployment.

4.13.1 Hash algorithms

BLAKE3 is selected as the primary hash algorithm. It achieves the highest measured throughput (294 MiB/s at 50MB), benefits from hardware SIMD parallelism, and delivers 7.3% higher throughput than SHA-256 at large file sizes with negligible differences at small sizes.

SHA-256 is selected for interoperability. It is universally supported by cloud key management services (AWS KMS, Google Cloud KMS, Azure Key Vault), X.509 certificate chains, and content delivery infrastructure. Its performance is competitive (274 MiB/s at 50 MB, accelerated by SHA-NI on x86 and the ARMv8 crypto extension on ARM platforms), and it functions as the primary compatibility baseline for any hybrid deployment interoperating with current infrastructure.

Keccak-256 is excluded. It's 39% throughput deficit relative to BLAKE3, and the lack of mainstream hardware acceleration provides no compensating advantage for cloud file-signing workloads. Keccak-256 would be reconsidered only in contexts calling for Ethereum-compatible hashing or SHA-3 standardisation mandates.

4.13.2 Classical signature schemes

EdDSA (Ed25519) is selected for Phase 2. As a primary classical scheme, it delivers the fastest signing (0.25 ms) and the smallest classical public-key signature (63 bytes) [53]. Its 0.841 ratio relative to RSA-PSS at 10 KB confirms its advantage. EdDSA appears to be the most suitable choice for new signing services.

ECDSA (P-256) is selected for Phase 2. It has superior compatibility as it is the most widely deployed asymmetric signature algorithm in cloud key management infrastructure and X.509 ecosystems. It is also required for

interoperability with existing hardware security modules and managed signing services. The signing performance (0.5 ms) and signature size (69 bytes) are competitive, so its slight disadvantage relative to EdDSA is not a concern.

RSA-PSS is excluded from Phase 2. Having a $20\times$ higher signing cost than EdDSA provides no benefit in this prototype. It remains in the evaluation as a historical baseline but is not recommended for any new deployment due to its security and performance.

HMAC-SHA256 is excluded due to the threat model scope. Despite the performance, as a symmetric Message Authentication Code (MAC), it provides no public verifiability and does not supply the integrity required for cloud file origin assurance.

4.13.3 PQC schemes

FN-DSA (Falcon) is selected as the primary PQC scheme. It provides the best all-round profile of any signature scheme in the study: the fastest public-key verification (0.25 ms), competitive signing (0.42 ms), the smallest PQC signature (654 bytes, equivalent to only 6.4% overhead on 10 KB objects), and an end-to-end ratio of 0.851 relative to RSA-PSS at 10 KB. Standardised as FN-DSA in FIPS 206 [57], Falcon completes the NIST post-quantum signature portfolio alongside ML-DSA and SLH-DSA.

ML-DSA (CRYSTALS-Dilithium) is selected as the secondary PQC scheme on grounds of primacy in standardisation. As the NIST-recommended post-quantum digital signature algorithm, it offers the broadest interoperability and performance, and is already deployed in TLS-integrated proof-of-concept environments by major cloud providers. Its signing cost (0.8 ms) and verification cost (0.39 ms) are competitive with the classical baseline, and its larger signature size (3.3 KB) is an acceptable trade-off for the quantum security it provides. ML-DSA signing is deterministic and avoids the floating-point Gaussian sampler used by FN-DSA, thereby reducing the complexity of side-channel analysis in constrained server environments.

SLH-DSA is excluded. Its 1 600 ms signing latency is disqualifying for any cloud file-signing workload involving online requests, and its 7.9 KB signature imposes a 77% storage amplification penalty on 10 KB objects. SLH-DSA may be included in future work specifically examining low-rate archival signing scenarios.

Table 9: Phase 2 carry-forward selection summary.

Category	Selected	Sign (ms)	Rationale
Hash	SHA-256	-	Interoperability + HW-accelerated
Hash	BLAKE3	-	Highest throughput + SIMD-native
Classical	EdDSA	0.25	Fastest + smallest + deterministic
Classical	ECDSA	0.50	KMS / X.509 interoperability
PQC	FN-DSA	0.42	Best sign + verify + size profile
PQC	ML-DSA	0.81	NIST primary standard + deterministic

4.14 Threats to Validity

This study has several threads to validity. While the aim of the project was to produce and test a prototype resembling cloud infrastructure, limitations apply and therefore pose a risk that these results will be considered less valid.

4.14.1 Localhost environment

Measurements were recorded on a single-host Docker deployment running over a loopback network interface. Network latency between services is therefore close to zero. Meanwhile, in a production cloud deployment, calls between services cross a physical network, and load balancers add tens to hundreds of milliseconds of latency. The absolute end-to-end latency figures (e.g. 32 ms for FN-DSA at 10 KB) will be substantially higher in a real cloud environment; however, the relative comparisons between signature profiles and the server-side stage decompositions are unaffected, because every condition is subject to the same network floor. Comparisons of the form "FN-DSA is $0.85\times$ the end-to-end cost of RSA-PSS" are transferable to cloud deployments even though the absolute values are not.

4.14.2 Generalisation to cloud hardware

The platform used for benchmarking operates on an general consumer AMD Ryzen 7 6800H CPU, unlike cloud service providers, that commonly deploy AMD EPYC and Intel Xeon server processors. The Ryzen 7's Advanced Vector Extensions 2 (AVX2) and Secure Hash Algorithm New Instructions (SHA-NI) sets are present on all modern server CPUs. Hence, the ranking of hash algorithms is expected to hold. On ARM Graviton instances, SHA-256's ARMv8 cryptographic extensions provide stronger acceleration than BLAKE3, potentially narrowing the current throughput gap observed here. Signature schemes costs are determined by the same algorithms on any 64-bit platform.

The only differences are potentially the frequency and microarchitectural pipeline depth, which affect all profiles proportionally.

4.14.3 Warm storage state only

The formal campaign was exclusively executed in warm mode for more uniform results. Cold-mode runs were not completed, so the data does not capture the additional I/O latency associated with the first access to each file. Cold results can be generated using the same campaign tooling and are expected to produce a wider spread in absolute numbers at small payload sizes, with cache effects that increase proportionally. That said, they are unlikely to alter the rank ordering of signature profiles.

4.15 Summary

Both FN-DSA and ML-DSA exhibit lower end-to-end latency than RSA-PSS across all payload sizes, notably with 10 KB ratios of 0.85 and 0.88, respectively. Except for SLH-DSA, post-quantum schemes have acceptable overheads. SLH-DSA is an outlier here, not because of post-quantum cryptography in general, but due to the specific characteristics of its hash-based construction. Hash latency is a negligible fraction of overall latency for small files, but it dominates for large files. In such a scenario, BLAKE3's 7% throughput advantage over SHA-256 becomes substantial. Keccak-256 consistently performs the worst among hashes with no compensating advantage. The quality gate results demonstrate that the harness produces measurements of acceptable stability for further decisions based on the evidence. Table 10 maps the key findings to their corresponding research questions.

Table 10: Summary of key findings mapped to research questions.

RQ	Finding	Evidence
RQ1	FN-DSA and ML-DSA impose less end-to-end latency than RSA-PSS across all payload sizes	Ratios: 0.85, 0.88 (95% CI)
RQ1	SLH-DSA are unsuitable for interactive signing	43.7× RSA-PSS at 10 KB
RQ1	Storage amplification, not latency, is the primary PQC cost for small objects	ML-DSA: 32% at 10 KB
RQ2	Hybrid configurations formally operate securely within the classical RSA-PSS execution bounds.	Measured Phase 2 EdDSA+FN-DSA $\approx$ 0.67 ms
RQ3	The harness produces stable, reproducible measurements	99.998% success; CV $\leq$ 7.5%
RQ4	PQC integration introduces no new vulnerability classes	STRIDE analysis (§3)

5 Discussion

5.1 Answering the Research Questions

The answers to each research question, in the context of the results presented in Section 4, are as follows.

5.1.1 RQ1: What is the performance cost of post-quantum and hybrid signature schemes compared to classical signatures in a complete end-to-end file-processing lifecycle?

Surprisingly, 2 out of 3 NIST-standardised post-quantum signature schemes clocked less latency than RSA-PSS, the most widely deployed classical scheme, across all payload sizes. FN-DSA achieves a ratio of 0.85 relative to RSA-PSS at 10 KB (95% CI [0.84, 0.86]), and ML-DSA achieves 0.88 (95% CI [0.87, 0.88]). This is a rather unexpected shift in the commonly assumed penalty for PQC results, driven by two factors. RSA-PSS private-key operations are inherently expensive, measured at 4.7-5.3 ms, due to modular exponentiation, while both lattice-based schemes are cheaper. Additionally, the dominant cost in the workflow at small file sizes is a 15-18 ms network and I/O floor that is common across conditions and masks algorithmic differences.

An important qualifier to this statement is that SLH-DSA is an outlier. It has a 1.6-second signing cost, producing a 43.7 ratio at 10 KB. PQC does not automatically imply performance penalties, so it is likely a consequence of the specific design decision to build a hash-based signer on top of Merkle trees. The SLH-DSA result shows that the critical decision is choosing an algorithm from the post-quantum portfolio, rather than considering post-quantum cryptography as a whole.

Storage-wise, ML-DSA and SLH-DSA impose 32% and 77% storage amplification on 10 KB objects, respectively, thus hiding this dimension from the latency benchmarking but clearly visible at cloud scale. In contrast, FN-DSA imposes only 6.4% storage overhead, making it the only post-quantum scheme competing with RSA-PSS on both latency and storage.

These findings complement and extend the algorithms benchmarks published by the Open Quantum Safe (OQS) project. OQS reports raw signing latencies consistent with the server-side primitive timings measured here, confirming that the implementation performance is in line with reference libraries. The contribution of this study is the demonstration that these costs of the cryptographic functions are largely absorbed by the shared infrastructure. This would otherwise have been invisible because microbenchmark methodologies do not include non-compute operations.

5.1.2 RQ2: What is the proven end-to-end performance footprint of deploying hybrid signature architectures, and what do these findings show for the viability of hybrid transition strategies?

Crucially, Phase 2 empirical datasets formally supersede previously projected theoretical telemetry bounds. By directly executing the hybrid combinations identified in Phase 1, hybrid execution was definitively proven to operate smoothly without disrupting core services.

Under sequential dual-signing, the combined measured cryptographic cost of the optimal hybrid pair (EdDSA + FN-Dsa) is highly efficient, achieving execution approximately $7\times$ faster than RSA-PSS alone (4.7 ms). Even the more conservative pairing (ECDSA + ML-Dsa) yields a system response that operates well below the RSA-PSS latency baseline. These empirical results establish a concrete operational measurement confirming systemically that hybrid configurations are immediately operationally viable. The formal Phase 2 execution confirms these projections with full system-level accounting, answering RQ2 natively.

5.1.3 RQ3: What reproducibility criteria must a system-level cryptographic benchmark satisfy, and does the harness developed in this study meet them?

The quality gate was developed to ensure the validity of the results. This has been successful in providing an operational answer. The benchmarking harness achieved 99.998% run success across 63 000 measured runs. Server telemetry coverage scored a full 100% and a median coefficient of variation below 7.5% across all conditions (except SLH-Dsa, which was excluded). Such achievement is attributable to the architectural choices discussed above. This included monotonic timers in the benchmark CLI that are immune to wall clock skew, as well as an appropriate inter-run delay, which is observed to be sufficient for OS scheduler queue drainage and connection pool recycling. The same applies to warm-up runs per condition, which obviate cold-start bias, and to seeded pseudorandom condition ordering, which prevents systematic order effects and enables perfect reproduction.

5.1.4 RQ4: What are the security implications and trade-offs of integrating PQC and hybrid signatures into an existing backend microservice architecture?

The security hardening phase leads to three conclusions. First, integrating post-quantum signature primitives into a microservice architecture does not

lead to new vulnerability classes; all vulnerabilities considered-path traversal, null injection, TOCTOU, and timing side channels on authentication-are generic to any file-based microservice and independent of the underlying cryptographic scheme. Second, the nonrepudiation property of public-key post-quantum signature schemes such as FN-DSA and ML-DSA aligns with the audit-trail integrity requirement identified in the literature review. These two schemes work the same way as the classical ones, i.e., signing and verifying manifests in the manifest-signature-and-verify workflow. Third, the role-based access control and audit logging mechanisms implemented here represent the operational layer that transforms cryptographic guarantees into auditable compliance evidence - a consideration that is frequently absent from algorithm-level benchmarks but essential for production deployment.

The HNDL risk model from Section 2, therefore, needs to be explicitly revisited. Indeed, the long-lived signing keys used in the prototype are exactly the assets exposed to retrospective compromise. However, when a manifest is signed with ML-DSA or FN-DSA, in addition to, or instead of, a classical key, the prototype produces provenance records whose integrity guarantees remain intact even if classical public-key cryptography is broken in the future. This provides the pragmatic driver for PQC migration in file-signing contexts: quantum-resistant signatures on long-lived records hedge against a real risk in the future; this is not about classical signing being too slow or about PQC being superior in every way from the outset.

5.2 What This Means for Practitioners

From a practitioner’s perspective, these findings imply the following three actions in assessing the migration of backend file-signing services to PQC:

There is a stronger performance argument for PQC migration than generally assumed. In particular, in the most important use case - a file-signing workflow at the system level - the data refutes the commonly assumed premise that moving to PQC entails a performance trade-off. Both FN-DSA and ML-DSA outperform RSA-PSS in an end-to-end file-signing workflow, and FN-DSA is even faster than ECDSA at signing. Thus, an organisation using RSA-PSS for manifest signing today can replace it with FN-DSA to achieve quantum resistance and reduce latency in a single step. One caveat applies: FN-DSA’s Gaussian trapdoor sampler requires careful constant-time implementation to resist side-channel attacks, and production deployments should validate the side-channel properties of their chosen FN-DSA library before adoption (see Section 5.5). The only PQC scheme that suffers a performance penalty is SLH-DSA, which is so severe that it should not be considered for any interactive signing path. However, SLH-DSA’s security profile may make

it suitable for use cases such as root certificate signing or long-term archival signatures, where the signing cost is amortised over a long verification horizon.

Storage amplification is the real cost of PQC, not latency. At cloud scale, the cost that grows proportionally with object count is not signing latency (which is amortised over the I/O floor) but signature bytes appended to every manifest. For a service storing millions of small objects (configuration records, telemetry events, audit entries), ML-DSA’s 3.3 KB signature adds measurable storage and cost. FN-DSA’s 654-byte signature is the only post-quantum option that keeps this cost in the same order of magnitude as that of classical alternatives. Organisations should factor in the expected object size distribution when selecting their algorithms, not just signing throughput.

The hybrid transition path is viable. The combined server-side measured signing cost of the best hybrid pair (EdDSA at 0.25 ms + FN-DSA at 0.42 ms $\approx$ 0.67 ms) is still decisively faster than RSA-PSS alone (4.7 ms). Even in the case of dual ML-DSA and ECDSA (0.81 + 0.50 ms $\approx$ 1.3 ms), the combined cost is well below RSA-PSS and negligible relative to the I/O floor at any payload size above 100 KB. This formally validates that the hybrid configurations securely isolated for Phase 2 are highly operationally deployable, a conclusion fully verified by the successful execution of the Phase 2 empirical campaign across full system-level accounting.

5.3 Balancing Algorithm Trade-offs

Selecting a post-quantum algorithm requires a comprehensive, personalised trade-off analysis that balances latency, storage, and security confidence at a cost-effective level.

Despite SLH-DSA being dropped from hybrid evaluations, it is an excellent choice for systems that prioritise maximum security, such as offline root certificate generation or long-term software attestation. It relies purely on well-understood hashing principles rather than newer lattice-based math, swapping performance for long-term peace of mind. Conversely, for standard enterprise workloads where network limits and storage costs are not restrictive, ML-DSA is the natural default because it is the primary NIST standard and gives broad compatibility. Finally, FN-DSA offers the greatest advantage in high-throughput environments. This is because it has a very low latency and a small signature size. However, because FN-DSA relies on complex floating-point operations that are harder to protect against side-channel attacks, it requires a more careful implementation. In short, no single algorithm fits every requirement, so engineering teams must choose the one that best suits their system constraints.

When these factors are considered together, low latency and small artefact

sizes are the ideal case. EdDSA + FN-DSA sits at this limit and offers the best total performance-size profile of any evaluated hybrid configuration.

5.4 Contributions

This dissertation makes four contributions to the body of knowledge on post-quantum cryptography in cloud systems.

1. An open, reproducible system-level benchmark for PQC file signing. The prototype and campaign tooling are fully documented and parameterised, which enables teams to reproduce the exact campaign or extend it with additional functionality using the same harness. This fills the reproducibility gap identified in the literature review.
2. Evidence that FN-DSA and ML-DSA impose no latency overhead relative to RSA-PSS in a complete cloud file-signing workflow, which disproves the common misconception that PQC adoption necessarily trades performance for security. The results of this evaluation provide a basis for migration recommendations.
3. A characterisation of storage amplification as the primary PQC cost dimension for small-object workloads. The observation that ML-DSA imposes a 32% storage amplification on 10 KB objects in primitive benchmarks is not prevalent because the signature size is deterministic, but it is a clue for future refactoring planning. This project is pioneering the estimates in the context of a realistic file-handling service.
4. A two-phase benchmark design that propagates Phase 1 evidence into Phase 2 selection. Rather than exhaustively testing all hybrid combinations (21 pairings), an optimised strategy has been developed. It reduces the campaign matrix to a tractable size while ensuring that the selection is data-driven rather than assumed. The two-phase design focuses effort on the four configurations with the highest viability, as supported by numerical evidence.

5.5 Limitations

This dissertation has several limitations that bound the generalisability of its conclusions.

The benchmarking environment is a single-host localhost Docker deployment. Absolute latency numbers will be higher in a production setup where

network latency between services adds tens of milliseconds. Relative comparisons of the profiles are transferable, as the baseline remains the same across conditions, but absolute SLA projections from this data would require retesting in a production cloud environment.

Only one parameter set per algorithm is evaluated (ML-DSA-44, SLH-DSA-128s, Falcon-512), corresponding to NIST Security Level 1-2 targets. Higher security levels are available for each scheme, with proportionally larger signatures and somewhat higher signing costs; the rank ordering is expected to hold but has not been verified.

Formal security analysis, side-channel resistance, and fault-injection tolerance are out of scope. In particular, FN-DSA’s Gaussian trapdoor sampler requires careful constant-time handling of floating-point operations; the implementation used by `pqcrypto-falcon` [75] targets correctness, but its side-channel properties have not been independently audited. In production deployments, this warrants validation against the timing-attack requirements for HSM or KMS integration.

5.6 Future Work

With both Phase 1 and Phase 2 campaigns formally executed, several structural verification paths exist to successfully extend the core scope and applicability of the fundamental benchmarking metrics mapping quantum integration.

5.6.1 Cloud-native deployment

Re-executing the campaign against a real cloud deployment would replace the localhost I/O floor with real cloud latency and confirm whether the relative comparisons hold under actual network lag and service overhead. ARM Graviton instances are of particular interest given that SHA-256’s ARMv8 acceleration may narrow the gap to BLAKE3.

5.6.2 Cold-mode benchmarking

The cold-mode campaign was not executed. Running it would assess the I/O cold-start contribution and show whether cache effects benefit or penalise any profile at small file sizes.

5.6.3 Higher security parameter sets

Benchmarking a higher security level variants like ML-DSA-65, ML-DSA-87, Falcon-1024, and SLH-DSA-192 and -256 would extend the analysis to NIST

Security Levels 3 and 5, relevant for organisations with higher assurance requirements.

5.6.4 Key Management Service integration

The prototype manages keys as files. A realistic cloud deployment delegates key material to a KMS (AWS KMS, GCP Cloud KMS, Thales HSM). Measuring the overhead of remote key operations - including the round-trip latency of a KMS-backed sign call - would provide a more complete picture of the operational cost of PQC in a production-grade architecture.

5.6.5 SLH-DSA for archival signing

SLH-DSA's 1 600ms signing cost makes it detrimental for interactive use. Still, its strict hash-based security assumptions and quick verification may make it well-suited for low-rate work where signing is rare, such as code signing or long-term archival manifests. Evaluating SLH-DSA within a more targeted operational model would complement the Phase 1 results.

6 Conclusion

This report seeks to answer whether post-quantum signature schemes are viable for cloud file-signing workflows and, if so, at what cost. The answer to the first question is yes, with important qualifications about algorithm choice. For the second one, the cost is lower than widely assumed. FN-DSA and ML-DSA, the two lattice-based NIST standards, are less expensive end-to-end than the current industry default RSA-PSS. Their storage overhead is manageable when the right scheme is chosen for the target workload’s object size distribution.

The more significant insight is that, at the system level, the cryptographic primitive is rarely the constraint. The data plane accounts for the majority of latency across all payload sizes, except the smallest. The HNDL threat model should therefore drive the decision to migrate to post-quantum cryptography, rather than by a false assumption that the transition is performance-prohibitive.

The Mosca equation [44] frames the agenda. An organisation whose signed records must remain trustworthy for a long timeframe, and whose migration timeline is measured in years, faces a window in which today’s signing keys are already at risk of future compromise. This report presents data showing that migrating those signing keys to FN-DSA or ML-DSA is a decision whose performance cost is lower than the cost of not migrating.

References

- [1] Amazon Web Services. Validating CloudTrail log file integrity, 2023. URL <https://docs.aws.amazon.com/awscloudtrail/latest/userguide/cloudtrail-log-file-validation-intro.html>.
- [2] Amazon Web Services. Using server-side encryption with AWS KMS keys (SSE-KMS), 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingKMSEncryption.html>.
- [3] Amazon Web Services. Checking object integrity in Amazon S3, 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity.html>.
- [4] Amazon Web Services. Checking object integrity for data at rest in Amazon S3, 2024. URL <https://docs.aws.amazon.com/AmazonS3/latest/userguide/checking-object-integrity-at-rest.html>.

- [5] Amazon Web Services. Client-side encryption, 2024. URL <https://docs.aws.amazon.com/kms/latest/cryptographic-details/client-side-encryption.html>.
- [6] Amazon Web Services. AWS Graviton processors, 2024. URL <https://aws.amazon.com/ec2/graviton/>.
- [7] Arm Limited. ARMv8-A cryptography extension, 2024. URL <https://developer.arm.com/documentation/101432/latest/Functional-description/About-the-Cryptographic-Extension>.
- [8] Elaine Barker and Allen Roginsky. Transitioning the use of cryptographic algorithms and key lengths, March 2019.
- [9] Matt Bishop and Michael Dilger. Checking for race conditions in file accesses. *Computing Systems*, 9(2):131–152, 1996.
- [10] C. Bormann and P. Hoffman. Concise binary object representation (CBOR). RFC 8949, December 2020.
- [11] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH (preliminary version), July 2022. URL <https://eprint.iacr.org/2022/975>.
- [12] Google Cloud. Google cloud KMS now supports quantum-safe digital signatures, February 2025. URL <https://cloud.google.com/blog/products/identity-security/google-cloud-kms-now-supports-quantum-safe-digital-signatures>.
- [13] Docker, Inc. Docker Compose overview, 2024. URL <https://docs.docker.com/compose/>.
- [14] Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979. doi: 10.1214/aos/1176344552.
- [15] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman & Hall/CRC, New York, 1993. doi: 10.1201/9780429246593.
- [16] Evan You and Vite Contributors. Vite: Next generation frontend tooling, 2024. URL <https://vitejs.dev/>.
- [17] Pierre-Alain Fouque, Jeffrey Hoffstein, Paul Kirchner, Vadim Lyubashevsky, Thomas Pornin, Thomas Prest, Thomas Pulles, Christine van Vredendaal, William Whyte, and Zhenfei Zhang. Fn-dsa: Fast-Fourier lattice-based compact signatures over NTRU. NIST Post-Quantum Cryptography Round 3 Submission, 2020. URL https://fn_dsa-sign.info.

- [18] Yanina García-Ruiz, Bernhard Teufel, Pantelis Stanitsas, et al. aws-s3-integrity-check: an open-source bash tool to verify the integrity of a dataset stored on Amazon S3. *GigaByte*, 2023. doi: 10.46471/gigabyte.87.
- [19] Craig Gidney and Martin Ekerå. How to factor 2048 bit rsa integers in 8 hours using 20 million noisy qubits. *Quantum*, 5:433, 2021. doi: 10.22331/q-2021-04-15-433.
- [20] Google Cloud. Envelope encryption, 2024. URL <https://cloud.google.com/kms/docs/envelope-encryption>.
- [21] Google Cloud. Google Axion processors, 2024. URL <https://cloud.google.com/blog/products/compute/introducing-googles-new-arm-based-cpu>.
- [22] Thales Group. ML-DSA programming guide for luna HSM, November 2025. URL https://www.thalesdocs.com/gphsm/luna/7/docs/network/Content/sdk/extensions/pqc/ML-DSA_programming_guide.htm.
- [23] Lov K. Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the 28th Annual ACM Symposium on Theory of Computing*, pages 212–219, 1996. doi: 10.1145/237814.237866.
- [24] Sean Gulley, Vinodh Gopal, Kirk Yap, Wajdi Feghali, Jim Guilford, and Gil Wolrich. Intel SHA Extensions: New instructions supporting the secure hash algorithm on Intel architecture processors. Technical report, Intel Corporation, July 2013. URL <https://www.intel.com/content/dam/develop/external/us/en/documents/intel-sha-extensions-white-paper-402097.pdf>.
- [25] HashiCorp. Harvest now, decrypt later: Why today’s encrypted data isn’t safe forever, 2025. URL <https://www.hashicorp.com/blog/harvest-now-decrypt-later>.
- [26] Jeffrey Hoffstein, Jill Pipher, and Joseph H. Silverman. NTRU: A ring-based public key cryptosystem. In *Algorithmic Number Theory – ANTS-III 1998*, volume 1423 of *Lecture Notes in Computer Science*, pages 267–288, Berlin, 1998. Springer. doi: 10.1007/BFb0054868.
- [27] The White House. National security memorandum on promoting united states leadership in quantum computing while mitigating risks to vulnerable cryptographic systems, May 2022. URL <https://www.whitehouse.gov/briefing-room/statements-releases/2022/05/04/national-security-memorandum-on-promoting-united-states-leadershi>

p-in-quantum-computing-while-mitigating-risks-to-vulnerabl
e-cryptographic-systems/. NSM-10.

- [28] A. Huelsing, D. Butin, S. Gazdag, J. Rijneveld, and A. Mohaisen. XMSS: extended merkle signature scheme. RFC 8391, May 2018.
- [29] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ECDSA). *International Journal of Information Security*, 1(1):36–63, 2001. doi: 10.1007/s102070100002.
- [30] Karen Kent and Murugiah Souppaya. Guide to computer security log management. NIST Special Publication 800-92, September 2006. URL <https://csrc.nist.gov/pubs/sp/800/92/final>.
- [31] Paul C. Kocher. Timing attacks on implementations of Diffie–Hellman, RSA, DSS, and other systems. In *Advances in Cryptology – CRYPTO 1996*, volume 1109 of *Lecture Notes in Computer Science*, pages 104–113. Springer, 1996. doi: 10.1007/3-540-68697-5_9.
- [32] H. Krawczyk, M. Bellare, and R. Canetti. HMAC: Keyed-hashing for message authentication. RFC 2104, February 1997.
- [33] Leslie Lamport. Constructing digital signatures from a one-way function. Technical Report SRI-CSL-98, SRI International Computer Science Laboratory, 1979.
- [34] Vadim Lyubashevsky. Lattice signatures without trapdoors. In *Advances in Cryptology – EUROCRYPT 2012*, volume 7237 of *Lecture Notes in Computer Science*, pages 738–755, Berlin, 2012. Springer. doi: 10.1007/978-3-642-29011-4_43.
- [35] J. Massimo, P. Kampanakis, S. Turner, and B. Westerbaan. Internet X.509 public key infrastructure: Algorithm identifiers for ML-DSA. RFC 9881, October 2025.
- [36] J. Massimo et al. Internet X.509 public key infrastructure: Algorithm identifiers for Kyber, 2024. URL <https://datatracker.ietf.org/doc/draft-ietf-lamps-kyber-certificates/>. IETF Internet-Draft.
- [37] Nicholas D. Matsakis and Felix S. Klock. The Rust language. In *Proceedings of the 2014 ACM SIGAda Annual Conference on High Integrity Language Technology*, pages 103–104. ACM, 2014. doi: 10.1145/2692956.2663188.

- [38] D. McGrew, M. Curcio, and S. Fluhrer. Leighton-micali hash-based signatures. RFC 8554, April 2019.
- [39] Dirk Merkel. Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239):2, 2014.
- [40] Ralph C. Merkle. A certified digital signature. In *Advances in Cryptology – CRYPTO 1989*, volume 435 of *Lecture Notes in Computer Science*, pages 218–238, Berlin, 1989. Springer. doi: 10.1007/0-387-34805-0_21.
- [41] Meta Platforms, Inc. React: The library for web and native user interfaces, 2024. URL <https://react.dev/>.
- [42] MinIO, Inc. MinIO object storage for Linux, 2024. URL <https://min.io/docs/minio/linux/index.html>.
- [43] MinIO, Inc. Minio archiving open source repository, February 2026. URL <https://github.com/minio/minio>.
- [44] Michele Mosca. Cybersecurity in an era with quantum computers: Will we be ready? *IEEE Security & Privacy*, 16(5):38–41, 2018. doi: 10.1109/MSP.2018.3761723.
- [45] National Institute of Standards and Technology. SHA-3 standard: Permutation-based hash and extendable-output functions. FIPS PUB 202, August 2015.
- [46] Nginx, Inc. Nginx documentation, 2024. URL <https://nginx.org/en/docs/>.
- [47] Jack O’Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O’Hearn. BLAKE3: One function, fast everywhere. In *IACR Cryptology ePrint Archive*, 2020. URL <https://eprint.iacr.org/2020/1235>. Report 2020/1235.
- [48] National Cybersecurity Center of Excellence. Migration to post-quantum cryptography, 2023. URL <https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms>.
- [49] National Institute of Standards and Technology. Announcing request for nominations for public-key post-quantum cryptographic algorithms, December 2016. URL <https://csrc.nist.gov/news/2016/public-key-post-quantum-cryptographic-algorithms>. Federal Register Notice.

- [50] National Institute of Standards and Technology. Post-quantum cryptography: Round 2 submissions, January 2019. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/round-2-submissions>.
- [51] National Institute of Standards and Technology. Post-quantum cryptography: Third round candidates, July 2020. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/round-3-submissions>.
- [52] National Institute of Standards and Technology. Recommendation for stateful hash-based signature schemes, October 2020.
- [53] National Institute of Standards and Technology. Digital signature standard (dss), February 2023.
- [54] National Institute of Standards and Technology. Module-lattice-based key encapsulation mechanism standard, August 2024.
- [55] National Institute of Standards and Technology. Module-lattice-based digital signature standard, August 2024.
- [56] National Institute of Standards and Technology. Stateless hash-based digital signature standard, August 2024.
- [57] National Institute of Standards and Technology. FN-DSA: FN-DSA-based digital signature algorithm, August 2024.
- [58] National Institute of Standards and Technology. Transition to post-quantum cryptography standards, November 2024.
- [59] National Institute of Standards and Technology. Post-quantum cryptography: Workshops and timeline, 2024. URL <https://csrc.nist.gov/projects/post-quantum-cryptography/workshops-and-timeline>.
- [60] National Institute of Standards and Technology. NIST selects HQC as fifth algorithm for post-quantum encryption, March 2025. URL <https://www.nist.gov/news-events/news/2025/03/nist-selects-hqc-fifth-algorithm-post-quantum-encryption>.
- [61] National Institute of Standards and Technology. Status report on the fourth round of the nist post-quantum cryptography standardization process, March 2025.
- [62] Christian Paquin, Douglas Stebila, and Goutam Tamvada. Benchmarking post-quantum cryptography in TLS. In *Post-Quantum Cryptography – PQCrypto 2020*, volume 12100 of *Lecture Notes in Computer Science*, pages 72–91. Springer, 2020. doi: 10.1007/978-3-030-44223-1.5.

- [63] PostgreSQL Global Development Group. PostgreSQL documentation: JSON types, 2024. URL <https://www.postgresql.org/docs/current/datatype-json.html>.
- [64] John Preskill. Quantum computing in the NISQ era and beyond. *Quantum*, 2:79, 2018. doi: 10.22331/q-2018-08-06-79.
- [65] Red Sift. State of post-quantum cryptography in pki, 2026. URL <https://redsift.com/resources/post-quantum-pki-progress>.
- [66] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. *Journal of the ACM*, 56(6):34:1–34:40, 2009. doi: 10.1145/1568318.1568324.
- [67] Ronald L. Rivest, Adi Shamir, and Leonard Adleman. A method for obtaining digital signatures and public-key cryptosystems. *Communications of the ACM*, 21(2):120–126, 1978. doi: 10.1145/359340.359342.
- [68] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2): 38–47, 1996. doi: 10.1109/2.485845.
- [69] Sectigo. Harvest now, decrypt later: Quantum threat, 2025. URL <https://www.sectigo.com/resource-library/harvest-now-decrypt-later-quantum-threat>.
- [70] Amazon Web Services. ML-KEM post-quantum TLS now supported in AWS KMS, ACM, and secrets manager, April 2025. URL <https://aws.amazon.com/blogs/security/ml-kem-post-quantum-tls-now-supported-in-aws-kms-acm-and-secrets-manager/>.
- [71] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997.
- [72] Adam Shostack. *Threat Modeling: Designing for Security*. Wiley, Indianapolis, IN, 2014.
- [73] Dimitrios Sikeridis, Panos Kampanakis, and Michael Devetsikiotis. Assessing the overhead of post-quantum cryptography in TLS 1.3 and SSH. In *Proceedings of the 16th International Conference on Emerging Networking Experiments and Technologies (CoNEXT 2020)*, pages 149–156. ACM, 2020. doi: 10.1145/3386367.3431305.

- [74] Douglas Stebila and Michele Mosca. Post-quantum key exchange for the Internet and the Open Quantum Safe project. In *Selected Areas in Cryptography – SAC 2016*, volume 10532 of *Lecture Notes in Computer Science*, pages 14–37. Springer, 2017. doi: 10.1007/978-3-319-69453-5_2.
- [75] The pqcrypto Authors. pqcrypto-falcon: Rust bindings to PQClean’s Falcon implementation, 2024. URL <https://crates.io/crates/pqcrypto-falcon>.
- [76] The Rand Project Developers. rand: Rust random number generation, 2024. URL <https://crates.io/crates/rand>.
- [77] The Rust Project Developers. std::time::instant — Rust standard library, 2024. URL <https://doc.rust-lang.org/std/time/struct.Instant.html>.
- [78] Jonathan S. Turner. New directions in communications (or which way to the information age?). *IEEE Communications Magazine*, 24(10):8–15, 1986. doi: 10.1109/MCOM.1986.1092946.
- [79] Yandex Cloud. Envelope encryption, 2024. URL <https://yandex.cloud/en/docs/kms/concepts/envelope>.

Appendices

A Supplementary Submission Material

This appendix block is reserved for formal review outputs, project-management evidence, and any additional artefacts required by the final submission regulations. In the final submission package, this section should include the second-marker review form, diary sheets, and any supplementary design or user documentation, all of which should be included as appendices.